VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi – 590018, Karnataka

A Major Project Phase I (ISEP23605) Report on

"CLOUD-BASED AUDITING SYSTEM WITH EFFICIENT OWNERSHIP MANAGEMENT FOR GROUP DATA TRANSACTIONS"

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

INFORMATION SCIENCE AND ENGINEERING

Submitted by

SANJAY G P — 1GA23IS141

SHASHANK GOWDA T K — 1GA23IS144

SRUJANGOUDA MALIPATIL — 1GA23IS165

YASHWANTH GOWDA D K — 1GA23IS188

Under the guidance of

[Guide Name]

[Designation], Dept. of ISE

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

GLOBAL ACADEMY OF TECHNOLOGY

RR Nagar, Bengaluru – 560098

Academic Year 2026–27

CERTIFICATE

[Certificate page — to be printed on the institute letterhead and signed]

This is to certify that the Major Project Phase I work entitled **"Cloud-Based Auditing System with Efficient Ownership Management for Group Data Transactions"** is a bonafide work carried out by **Sanjay G P (1GA23IS141), Shashank Gowda T K (1GA23IS144), Srujangouda Malipatil (1GA23IS165)** and **Yashwanth Gowda D K (1GA23IS188)**, students of the Department of Information Science and Engineering, Global Academy of Technology, Bengaluru, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Information Science and Engineering of Visvesvaraya Technological University, Belagavi, during the academic year 2026–27. It is certified that all corrections and suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Signature of the Guide [Guide Name]	Signature of the HOD [HOD Name]
Signature of the Principal [Principal Name]	External Viva — Examiners 1. 2.

DECLARATION

[Declaration page — to be signed by the candidates]

We, **Sanjay G P (1GA23IS141), Shashank Gowda T K (1GA23IS144), Srujangouda Malipatil (1GA23IS165)** and **Yashwanth Gowda D K (1GA23IS188)**, students of the Sixth Semester B.E., Department of Information Science and Engineering, Global Academy of Technology, Bengaluru, hereby declare that the Major Project Phase I work entitled **"Cloud-Based Auditing System with Efficient Ownership Management for Group Data Transactions"** has been carried out by us and submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Information Science and Engineering of Visvesvaraya Technological University, Belagavi, during the academic year 2026–27. We further declare that, to the best of our knowledge and belief, the work reported here does not form part of any other dissertation on the basis of which a degree or award was conferred earlier on any other candidate.

Sanjay G P (1GA23IS141)	Shashank Gowda T K (1GA23IS144)
Srujangouda Malipatil (1GA23IS165)	Yashwanth Gowda D K (1GA23IS188)

Place: Bengaluru
Date: _____

ABSTRACT

Cloud storage has become the standard medium for holding and sharing large datasets, yet two problems remain unresolved by ordinary software: verifying that outsourced data is still stored intact without downloading it, and transferring the ownership of that data between groups of users in a way that is cryptographically enforced rather than merely recorded. Existing Provable Data Possession (PDP) schemes largely assume a single owner, and the known data-transfer extensions require the seller to disclose a secret key to the buyer — an act that enables a collusion attack between the buyer and the cloud.

This project designs and implements a **cloud-based auditing system with efficient ownership management for group data transactions**, following the base scheme of Wu, You and Huang (IEEE Transactions on Information Forensics and Security, vol. 20, 2025). The system involves four entities — a Cloud Storage Provider (CSP), a Third-Party Auditor (TPA), a Sale Group (SG) and a Buy Group (BG) — and uses BLS aggregate signatures, homomorphic authenticators and random masking to support public integrity auditing, dynamic owner addition and revocation, and secure group-to-group ownership transfer through constant-size Ownership Dynamic Tokens.

All six algorithms of the scheme (SysGen, KeyGen, TagGen, Audit, OwnerModify and OwnerTransfer) were implemented in Python on the BLS12-381 pairing curve, exercised by 41 automated correctness tests, three executable attack demonstrations (collusion, rogue-key and tampering) and a benchmark suite reproducing the base paper's experimental programme. The measurements confirm the paper's central claims: key generation and membership changes scale linearly in the number of owners against the baseline's quadratic cost (10.5× faster at twenty owners), auditing cost is independent of group size, the ownership token is a constant 160 bytes, and both the collusion and rogue-key attacks that break the prior art fail against the implemented scheme.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

Cloud computing has become a widely used technology for storing and sharing large amounts of data due to its scalability, flexibility, and cost-effectiveness. With the rapid growth of artificial intelligence, big data, and online services, organizations increasingly rely on cloud storage to manage valuable information. However, ensuring the integrity and security of outsourced data remains a major challenge, especially when data is shared among multiple users or groups.

Traditional cloud auditing schemes mainly support single-owner environments and often fail to provide secure and efficient ownership transfer. Moreover, downloading the entire dataset for verification is impractical because it increases bandwidth usage and computational costs. To address these issues, cloud auditing techniques such as Provable Data Possession (PDP) enable users to verify data integrity without retrieving the complete data from the cloud.

This project proposes a *Cloud-based Auditing System with Efficient Ownership Management for Group Data Transaction*. The system supports secure data auditing, dynamic ownership management, and efficient ownership transfer between users or groups. Using cryptographic techniques such as BLS signatures and homomorphic authentication, the proposed framework ensures data integrity while protecting against attacks such as key leakage, collusion, unauthorized access, and data tampering. A Third-Party Auditor (TPA) performs integrity verification on behalf of users, reducing computational overhead and improving efficiency.

The proposed solution provides a secure, scalable, and reliable cloud auditing framework that supports modern cloud-based data-sharing applications. By enabling efficient ownership transfer and continuous integrity verification, it enhances trust, security, and performance in cloud storage environments.

1.1 Briefing About the Project

The development of this project follows a structured Software Development Life Cycle (SDLC) to ensure the successful design, implementation, and deployment of the system. The project is divided into five major phases: Requirement Analysis, Design, Development, Testing, and Release.

Requirement Analysis Phase: In this phase, the requirements of the system are identified and analyzed. The objectives include secure cloud data storage, efficient ownership transfer, dynamic user management, and reliable data integrity auditing. Existing cloud auditing

techniques and their limitations are studied to define the functional and security requirements of the proposed system.

Design Phase: During the design phase, the overall architecture of the system is prepared. The interactions among the Cloud Storage Provider (CSP), Third Party Auditor (TPA), Sale Group (SG), and Buy Group (BG) are defined. System modules such as key generation, tag generation, data auditing, ownership modification, and ownership transfer are designed using appropriate object-oriented and cryptographic principles. A good design improves extensibility, reusability, and maintainability while reducing future development costs.

Development Phase: In this phase, the designed modules are implemented using suitable programming languages and tools. Cryptographic algorithms such as BLS signatures and homomorphic authentication are integrated to provide secure auditing and ownership management. All system functionalities are developed and connected to form a complete working application.

Testing Phase: The developed system is thoroughly tested to verify its correctness, reliability, and security. Various test cases are executed to validate data integrity auditing, ownership transfer operations, and resistance against attacks such as data tampering, key leakage, and unauthorized access.

Release Phase: After successful testing, the system is deployed for practical use. The final product provides secure cloud auditing, efficient ownership management, and scalable data transaction capabilities. The release phase ensures that the system meets user requirements and operates effectively in a real-world cloud environment.

1.2 Existing System

- Existing cloud auditing systems are used to verify the integrity and availability of data stored in cloud environments.
- These systems allow users to check the correctness of outsourced data without downloading the entire dataset, reducing communication and storage overhead.
- The most existing solutions are designed for single-owner environments and have limited support for dynamic ownership management, secure ownership transfer, and group-based data transactions.
- Few of the existing system are:
    - Provable Data Possession (PDP)
    - Proof of Retrievability (POR)
    - Third Party Auditing (TPA) System
    - Existing Ownership Transfer Schemes

1.2.1 Provable Data Possession (PDP)

PDP enables users to verify the integrity of cloud-stored data without retrieving the entire file. It reduces communication and storage overhead through cryptographic verification techniques. However, traditional PDP schemes mainly support static data and single-owner systems, limiting their applicability in collaborative environments.

1.2.2 Proof of Retrievability (POR)

POR ensures that cloud-stored data is both intact and recoverable when required. It uses cryptographic proofs and error-correcting codes to guarantee data availability. Although it provides strong security, it introduces additional storage and computational overhead and lacks efficient ownership transfer mechanisms.

1.2.3 Third Party Auditing (TPA) System

TPA systems allow an external auditor to verify data integrity on behalf of cloud users. This reduces the burden on data owners and improves trust in cloud services. However, these systems mainly focus on auditing and do not effectively support dynamic ownership changes or group-based data transactions.

1.2.4 Existing Ownership Transfer Schemes

Existing ownership transfer schemes allow cloud data ownership to be transferred between users without downloading the data. While they improve data transaction efficiency, most schemes support only single-owner transfers and are vulnerable to issues such as high computation costs, collusion attacks, and key leakage in multi-owner environments.

1.3 Limitations of Existing System

- **Limited Multi-Owner Support:** Most cloud auditing schemes are designed for single-owner environments and do not efficiently support multiple users or groups.
- **Inefficient Ownership Transfer:** Ownership transfer requires high computation and communication overhead, reducing efficiency in large-scale systems.
- **Lack of Dynamic Management:** Adding or revoking users often requires complex key updates and system reconfiguration.
- **Security Vulnerabilities:** Existing schemes are prone to collusion attacks, key leakage, unauthorized access, and data tampering.
- **Poor Scalability:** System performance decreases as the number of users, groups, and data transactions increases.

1.4 Proposed System

The proposed system is a scalable cloud auditing protocol designed for secure data storage and efficient ownership transfer within group transactions. It addresses the fundamental conflict between fixed ownership distribution and the dynamic nature of group members while protecting against collusion and rogue-key attacks.

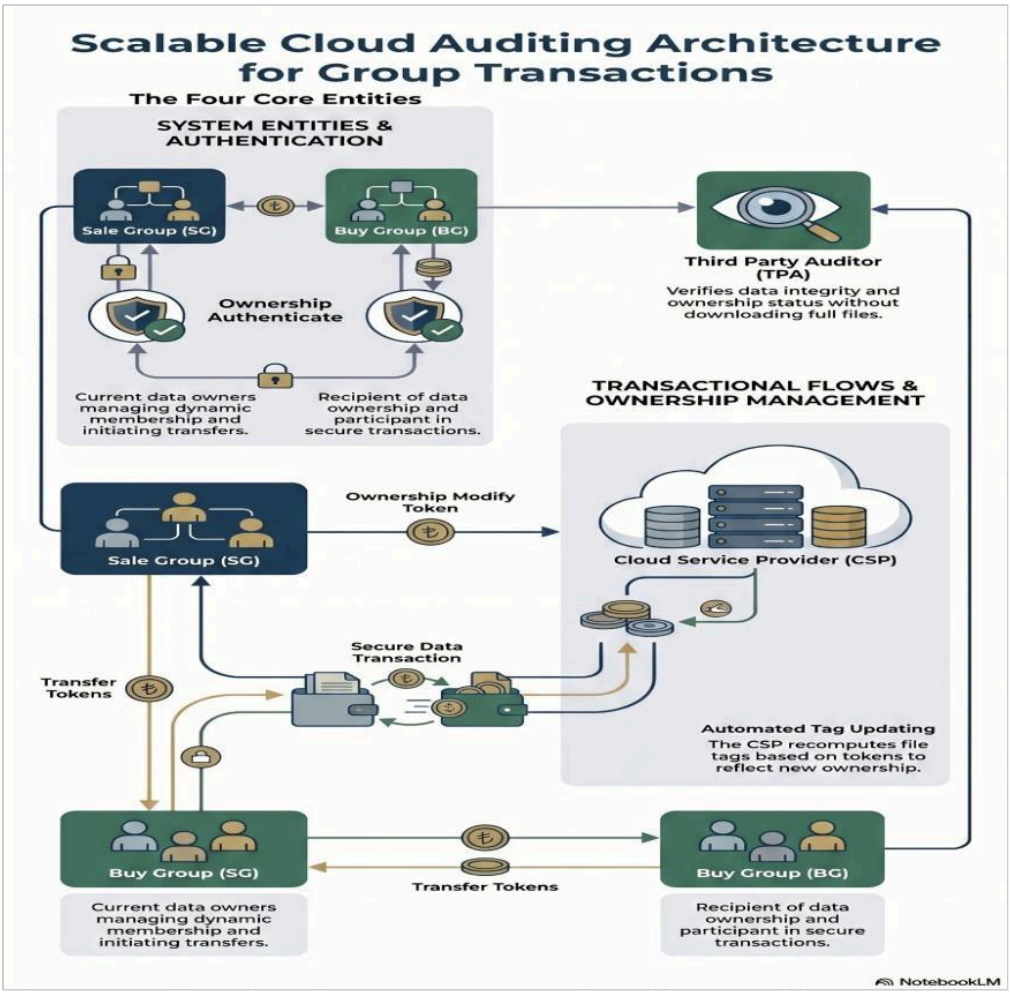


Figure 1.1: Proposed system

The proposed system is a secure and scalable cloud auditing framework designed for multi-owner data transactions. It involves four main entities: the Cloud Storage Provider (CSP), Third Party Auditor (TPA), Sale Group (SG), and Buy Group (BG). The framework enables secure data integrity verification without downloading the entire dataset and supports efficient partial or full ownership transfer among users. By using BLS signatures, aggregated public keys, and random masking techniques, the system protects against key leakage, collusion attacks, rogue-key attacks, and unauthorized data tampering. Its linear scalability makes it suitable for large-scale cloud storage and data-sharing environments.

1.5 Objectives

1. Study cloud auditing and data ownership techniques (PDP, POR, DT-PDP)
2. Design a scalable auditing framework (CSP, TPA, SG, BG)
3. Enable secure partial & full ownership transfer
4. Verify data integrity without full data download
5. Improve security against key leakage, collusion, and tampering

1.6 Motivation

- **Security Gap:** Key exposure during ownership transfer → collusion attack
- **Multi-Owner Limitation:** Inefficient handling of group changes
- **Scalability Issue:** High overhead ($O(s^2)$) for large groups
- **Key Leakage Risk:** Secret key transfer causes audit failures
- **Growing Demand:** Need for secure, transferable data ownership in AI datasets

CHAPTER 2

LITERATURE SURVEY

A literature survey is essential to understand the current state of research, identify limitations in existing solutions, and explore techniques that can be adapted to solve the identified problem. By studying previously published research papers, various auditing mechanisms, cryptographic techniques, and ownership management approaches can be analyzed and compared. The primary objectives of this literature survey are to examine existing cloud auditing schemes, investigate ownership transfer mechanisms, evaluate their security and performance, and identify research gaps that motivate the proposed system.

2.1 Introduction

Cloud computing has become one of the most important technologies for storing, managing, and sharing data over the Internet. Organizations and individuals increasingly rely on cloud storage services because they offer scalability, flexibility, and cost-effective data management. However, as data is shared among multiple users and groups, ensuring data integrity, secure ownership management, and efficient auditing becomes a major challenge. The proposed project, Cloud-Based Auditing System with Efficient Ownership Management for Group Data Transactions, aims to provide a secure and scalable solution for verifying data integrity while supporting dynamic ownership transfer among multiple users in cloud environments.

Studying previous research work is essential because it helps in understanding existing cloud auditing techniques, ownership transfer mechanisms, and security solutions. A literature survey provides insights into the strengths and limitations of current approaches, identifies research gaps, and highlights challenges related to scalability, privacy preservation, communication overhead, and resistance to security attacks. By analyzing earlier studies, researchers can build improved systems that overcome the shortcomings of existing solutions.

The primary objectives of this literature survey are to review recent developments in cloud auditing and ownership management, analyze various security and auditing techniques proposed by researchers, compare their performance and effectiveness, and identify limitations that motivate the proposed system.

2.2 Background of the Study

Cloud computing has transformed the way data is stored, accessed, and shared by providing on-demand services through the Internet. As organizations increasingly depend on cloud storage for managing large volumes of data, ensuring data integrity, confidentiality, and secure

ownership management has become a critical requirement. In cloud environments, multiple users often collaborate and share data, making it necessary to verify that stored data remains unaltered while supporting secure ownership transfer among users and groups.

The proposed project is based on the concepts of cloud auditing and ownership management. Cloud auditing enables users to verify the integrity of data stored on cloud servers without downloading the entire dataset. This is achieved through cryptographic techniques such as Provable Data Possession (PDP) and Proof of Retrievability (POR), which allow efficient verification of remote data storage. Public auditing mechanisms further enhance trust by enabling independent verification through a Third-Party Auditor.

- **Cloud Storage Provider (CSP):** An entity that stores user data and provides cloud storage services.
- **Third-Party Auditor (TPA):** An independent entity responsible for verifying the integrity of cloud-stored data.
- **Data Integrity:** The assurance that stored data remains accurate, complete, and unaltered.
- **Provable Data Possession (PDP):** A cryptographic technique that enables verification of data stored on remote servers.
- **Proof of Retrievability (POR):** A mechanism that ensures both data integrity and recoverability.
- **Ownership Transfer:** The process of securely transferring data ownership rights from one user or group to another.
- **Aggregate Signature:** A digital signature that combines multiple signatures into a single compact signature.
- **Homomorphic Authentication:** A technique that allows verification operations on authenticated data without revealing the actual content.
- **Rogue-Key Attack:** A security attack where malicious users manipulate public keys to forge valid signatures.
- **Collusion Attack:** Multiple entities collaborate to breach security.

2.3 Review of Existing Work

1. Dynamic Outsourced Auditing Services for Cloud Storage Based on Batch-Leaves-Authenticated Merkle Hash Tree — Lu Rao, Hua Zhang, Tengfei Tu
This paper proposes a dynamic cloud auditing mechanism using a Batch-Leaves-Authenticated Merkle Hash Tree. The scheme improves auditing efficiency and supports dynamic data operations. However, it suffers from high computational costs at the Third-Party Auditor and depends on the Bitcoin network for verification.

2. Dynamic-Hash-Table Based Public Auditing for Secure Cloud Storage — Hui Tian et al.

The authors introduce a Dynamic Hash Table (DHT)-based public auditing system that enables dynamic updates and privacy-preserving auditing. The scheme improves storage management but requires additional storage at the auditor side and assumes a trusted TPA.

3. Efficient Identity-Based Provable Multi-Copy Data Possession in Multi-Cloud Storage — Jiguo Li, Hao Yan, Yichen Zhang

This work uses identity-based cryptography and homomorphic tags to verify multiple copies of data stored across multiple clouds. While it reduces certificate management overhead, it incurs high computation and storage costs.

4. Efficient Public Verification on the Integrity of Multi-Owner Data in the Cloud — Boyang Wang et al.

The scheme uses multisignatures to support public verification of multi-owner cloud data. It provides secure auditing but requires signatures from all owners and introduces communication overhead during verification.

5. Enabling Efficient User Revocation in Identity-Based Cloud Storage Auditing for Shared Big Data — Yue Zhang et al.

This paper presents an identity-based auditing framework that supports efficient user revocation through key updates. It improves management of shared data but still depends on trusted entities such as PKG and TPA.

6. Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing — Qian Wang et al.

The authors use homomorphic authenticators and Merkle Hash Trees to support dynamic data updates and public auditing. The system improves security but involves complex cryptographic operations and communication overhead.

7. Experimental Evaluation of Secure and Verifiable Data Control Access over Cloud Storage Environment using Dynamic Cryptographic Strategy — T. Kavitha et al.

This work combines CP-ABE, deniable encryption, and NTRU cryptography to provide secure access control. Although security is enhanced, the overall system complexity and latency increase significantly.

8. Identity-Based Privacy Preserving Remote Data Integrity Checking for Cloud Storage — Jiguo Li, Hao Yan, Yichen Zhang

The paper introduces an identity-based remote data integrity checking mechanism using homomorphic tags and random masking. It ensures privacy preservation but requires expensive bilinear pairing operations.

9. Improved Security and Privacy in Cloud Data Security and Privacy: Measures and Attacks — Ashish Joshi et al.
This research surveys cloud security techniques such as AES, RSA, ECC, homomorphic encryption, and steganography. It provides strong security recommendations but increases computational complexity and system overhead.

10. PPADT: Privacy-Preserving Identity-Based Public Auditing with Efficient Data Transfer for Cloud-Based IoT Data — Chao Gai et al.
The proposed scheme supports identity-based auditing and efficient data transfer in IoT cloud environments. Privacy is preserved through pseudonyms and random masking, but large-scale implementation remains challenging.

11. Keyword-Based Remote Data Integrity Auditing Supporting Full Data Dynamics — Wenting Shen et al.
This work introduces keyword-based auditing that allows users to verify selected data efficiently. It supports full data dynamics but incurs additional setup costs and depends heavily on a TPA.

12. Privacy-Preserving Public Auditing for Shared Data in the Cloud — Boyang Wang, Baochun Li, Hui Li
The authors employ ring signatures and homomorphic authenticators to protect user identities during auditing. The scheme supports dynamic data operations but requires recomputation when group membership changes.

13. Provable Multicopy Dynamic Data Possession in Cloud Computing Systems — Ayad F. Barsoum, M. Anwar Hasan
The paper proposes a multicopy data possession framework supporting insert, delete, modify, and append operations. Communication and storage costs increase as the number of copies grows.

14. Provable Data Possession with Outsourced Data Transfer — Huaqun Wang et al.
This study introduces outsourced data transfer using DT-PDP and bilinear pairing cryptography. The method reduces owner burden but depends on trusted third parties and incurs communication overhead.

15. Secure and Proficient Provable Data Procurity with Privacy Protection in Cloud Storage — Dr. C. Siva Kumar et al.
The scheme provides privacy-preserving third-party auditing and batch verification for multiple owners. It improves privacy protection but still suffers from high computational costs.

16. Secure Deduplication and Cloud Storage Auditing with Efficient Dynamic Ownership Management and Data Dynamics — Xueqi Peng et al.
This work combines secure deduplication, cloud auditing, and dynamic ownership

management. It supports efficient storage utilization but introduces complexity through multiple indexing and management structures.

17. A Novel Efficient Remote Data Possession Checking Protocol in Cloud Storage — Hao Yan et al.
The authors propose a remote data possession protocol based on homomorphic hash functions and operation record tables. It supports dynamic operations but limits auditing to private verification.

18. An Efficient Methodology to Audit Data in Cloud Storage to Ensure Reliability and Integrity using Secured Crypto Schemes — Vijay Vasanth Aroulanandam et al.
The paper integrates Proof of Retrievability (PoR), Provable Data Possession (PDP), AES, and RSA encryption to improve cloud auditing reliability. However, auditing time and storage overhead remain high.

19. Blockchain-Based Accountable Auditing with Multi-Ownership Transfer — Jun Shen et al.
This research integrates blockchain technology with cloud auditing and multi-ownership transfer. It enhances transparency and accountability but introduces blockchain-related communication and storage overhead.

20. Certificateless Public Auditing for Data Integrity in the Cloud — Boyang Wang et al.
The authors propose a certificateless auditing mechanism using homomorphic authenticable signatures. The scheme eliminates certificate management but increases verification complexity compared to traditional methods.

2.4 Comparative Analysis

The most existing systems focus on ensuring data integrity and supporting public auditing. Several schemes provide privacy preservation through identity-based cryptography, ring signatures, homomorphic authenticators, and encryption techniques. Dynamic data operations such as insertion, deletion, and modification are supported in many approaches.

However, only a few papers provide efficient ownership management and ownership transfer capabilities. Blockchain-based solutions improve transparency and accountability but increase communication and storage costs. Identity-based and certificateless approaches reduce certificate management issues but still require expensive cryptographic computations.

1. Supporting efficient ownership transfer among multiple users and groups.
2. Providing scalable ownership management with dynamic owner addition and revocation.
3. Reducing communication overhead through batch auditing and aggregate signatures.

4. Eliminating dependence on fully trusted third parties.
5. Protecting ownership information using random masking techniques.
6. Resisting collusion attacks and rogue-key attacks.
7. Supporting partial ownership transfer without affecting existing owners.
8. Maintaining high auditing efficiency even for large-scale cloud environments.

2.5 Research Gap Identification

The literature survey reveals that significant progress has been made in cloud auditing, data integrity verification, privacy preservation, and secure cloud storage. However, several important challenges remain unresolved in existing research, creating the need for an improved cloud auditing framework.

1. Limited Support for Multi-Owner and Group-Based Environments
Most existing cloud auditing schemes are designed for single-owner data storage systems. Although some approaches support shared data, they do not efficiently handle ownership management among multiple users or groups. The proposed Cloud-Based Auditing System introduces a group-oriented ownership management mechanism that supports multiple owners and enables secure management of shared cloud data.

2. Inefficient Ownership Transfer Mechanisms
Existing ownership transfer schemes often involve expensive cryptographic operations, repeated signature generation, and high communication overhead. These factors reduce system efficiency, especially in large-scale cloud environments. The system uses BLS signatures and ODTs for efficient, low-cost ownership transfer.

3. Lack of Dynamic Ownership Management
Many surveyed papers support data auditing but do not provide effective mechanisms for dynamic owner addition, owner revocation, or ownership modification. User management often requires complete system reconfiguration. The proposed framework supports dynamic ownership operations, including owner addition, revocation, and modification, without requiring complete re-signing.

4. Dependence on Trusted Third Parties
Several existing approaches rely heavily on trusted entities such as Third-Party Auditors (TPAs), Public Key Generators (PKGs), or dealers for verification and ownership management. This creates security risks and single points of failure. The proposed system minimizes dependence on fully trusted third parties by using cryptographic verification mechanisms and secure ownership tokens, thereby improving reliability and security.

5. Vulnerability to Security Attacks
Many existing cloud auditing schemes remain vulnerable to collusion attacks, rogue-key

attacks, key leakage, and unauthorized ownership modifications. These threats can compromise both data integrity and ownership authenticity. The proposed framework incorporates random masking techniques, secure token generation, and robust authentication mechanisms to provide resistance against collusion attacks, rogue-key attacks, and unauthorized access.

6. High Computational and Communication Overhead

Several surveyed solutions experience significant performance degradation as the number of users, data blocks, and auditing requests increases. This limits their practical deployment in large-scale cloud systems. The project employs batch auditing and aggregate signature techniques to reduce computation and communication overhead, improving overall system performance and efficiency.

CHAPTER 3

SYSTEM REQUIREMENTS AND SPECIFICATION

The system requirements and specifications necessary for the successful implementation of the proposed Cloud-Based Auditing System with Efficient Ownership Management for Group Data Transaction. System requirements define the hardware and software resources needed to develop, deploy, and operate the application effectively. In addition, the chapter describes the functional and non-functional requirements that specify the expected behavior, performance, security, reliability, and usability of the system. These requirements serve as the foundation for designing a secure, scalable, and efficient cloud auditing framework capable of supporting dynamic ownership management and group data transactions.

3.1 Hardware Requirements

The minimum hardware requirements for the proposed system are:

Component	Specification
Processor	Intel Core i3 or above
RAM	8 GB minimum (16 GB recommended)
Hard Disk	256 GB SSD or above
System Architecture	64-bit
Network Connection	Stable Internet connection
Display	1366 × 768 resolution or higher

3.2 Software Requirements

The software requirements for implementing the proposed system are:

Component	Specification
Operating System	Windows 10/11, Linux, or macOS
Programming Language	Java / Python
Frontend	HTML, CSS, JavaScript
Backend Framework	Spring Boot / Django / Flask

Database	MySQL
Cloud Platform	Cloud Storage Server
Development Environment	VS Code / IntelliJ IDEA / Eclipse
Web Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox
Version Control	Git and GitHub
Security Libraries	BLS Signature Library, Cryptographic APIs

> **Note.** In the present phase of the project, the web deployment layer is implemented as a **Flask** web application: a Flask + SQLAlchemy web console now exists in the project alongside the cryptographic core, satisfying the backend-framework and database rows above.

3.3 Functional Requirements

Functional requirements describe the core functionalities that the system must perform.

1. **User Registration and Authentication:** The system shall allow users to register and securely log in using valid credentials.
2. **Data Upload and Storage:** Authorized users shall be able to upload files to the cloud storage server securely.
3. **Data Integrity Verification:** The system shall verify the integrity of stored data using cryptographic auditing techniques.
4. **Public Auditing:** The Third-Party Auditor (TPA) shall be able to perform auditing without accessing the actual content of the stored data.
5. **Ownership Transfer:** The system shall support secure transfer of data ownership between users or groups.
6. **Dynamic Ownership Management:** The system shall allow owner addition, owner revocation, and ownership modification without affecting existing owners.
7. **Batch Auditing:** The system shall support auditing multiple files and users simultaneously to reduce computation overhead.
8. **Security Monitoring:** The system shall detect unauthorized access attempts and maintain secure ownership records.
9. **Audit Report Generation:** The system shall generate audit reports showing the status of data integrity verification.
10. **Group Data Management:** The system shall support secure sharing and management of cloud data among multiple users and groups.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance expectations of the system.

1. **Security:** The system must ensure confidentiality, integrity, authentication, and authorization through cryptographic techniques.
2. **Scalability:** The system should efficiently handle a large number of users, files, ownership transfers, and auditing requests.
3. **Reliability:** The system should provide accurate auditing results and maintain consistent performance under varying workloads.
4. **Performance:** The auditing and ownership transfer processes should be completed with minimal computational and communication overhead.
5. **Maintainability:** The system should be designed to support future enhancements and modifications with minimal effort.
6. **Interoperability:** The system should operate effectively across different platforms, operating systems, and cloud environments.
7. **Privacy Preservation:** The system should protect sensitive user and ownership information during auditing and ownership transfer operations.
8. **Fault Tolerance:** The system should remain functional despite failures and unexpected conditions.

CHAPTER 4

SYSTEM DESIGN

The System Design phase translates the project's conceptual requirements into a structured architectural framework, bridging the gap between theoretical objectives and technical implementation. This chapter details the construction of a scalable cloud auditing framework designed to maintain data integrity and facilitate secure ownership management within dynamic group environments. By integrating cryptographic standards such as BLS signatures and homomorphic authenticators, the design ensures that the system can withstand various security threats, including collusion and rogue-key attacks, while optimizing computational and communication costs.

4.1 High Level Design

The High Level Design outlines the structural organization of the system, which is built around four primary entities: the Cloud Storage Provider (CSP), the Third Party Auditor (TPA), the Sale Group (SG), and the Buy Group (BG). The CSP acts as a semi-trusted entity that stores large datasets and facilitates external integrity verification, whereas the TPA is a powerful entity tasked with providing reliable auditing results to data owners. The SG and BG are ad-hoc groups of data owners where ownership is shared; the design specifically allows for dynamic group membership — permitting owners to be added or revoked — and enables the efficient transfer of ownership from a selling group to a buying group using specialized dynamic tokens. This multi-entity architecture ensures that data integrity is provable throughout its lifecycle, even as ownership changes hands between different organizational groups.

4.1.1 System Architecture

The System Architecture of this project is designed to facilitate secure and scalable data integrity auditing within a multi-owner environment. Unlike traditional single-owner schemes, this architecture specifically addresses the dynamic nature of group membership and the complexities of transferring ownership between distinct organizational groups. The framework utilizes BLS signatures and homomorphic authenticators to allow a Third Party Auditor to verify data without downloading the entire dataset, while random masks are integrated into file tags to prevent key leakage and protect against collusion attacks during transactions.

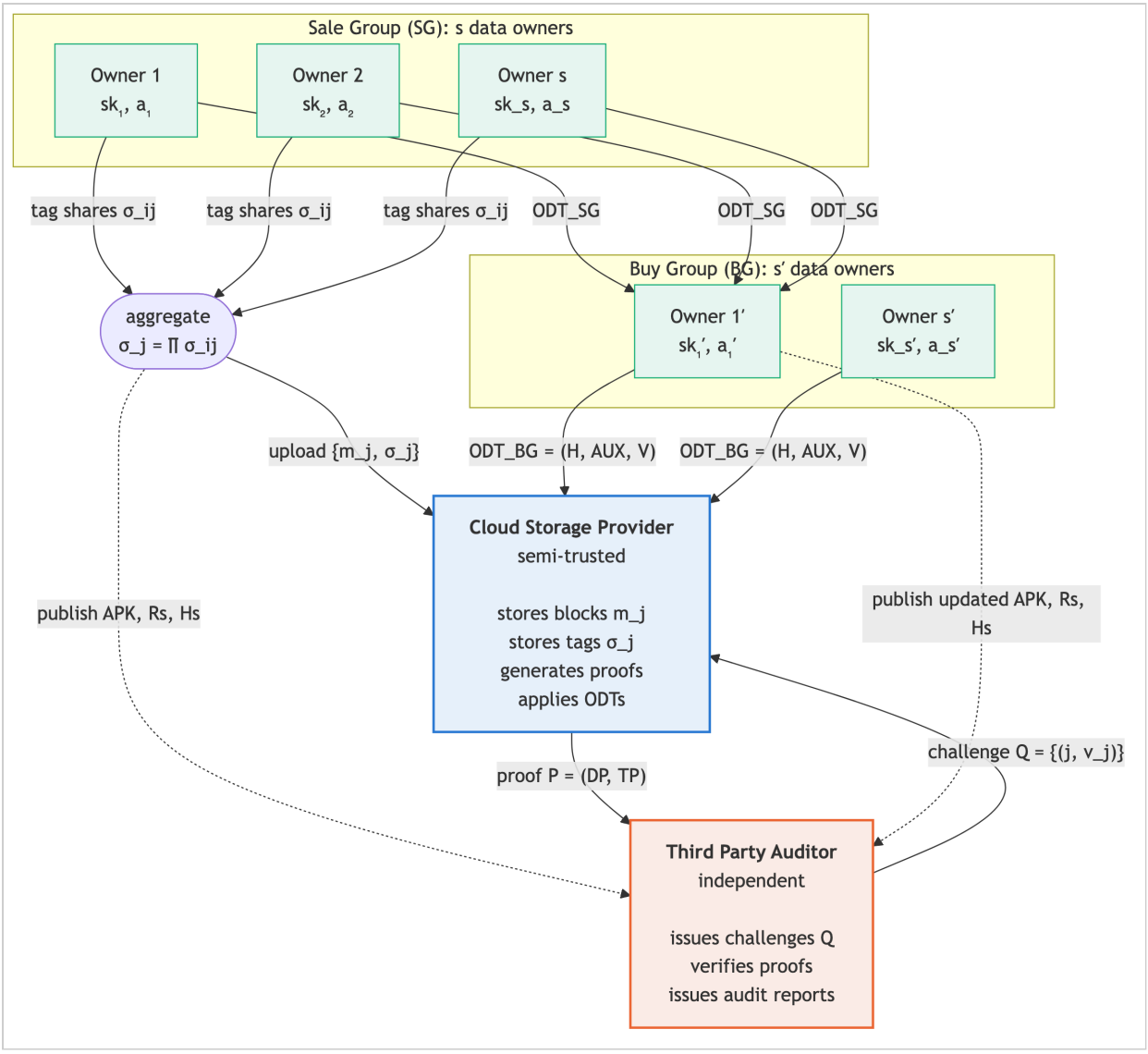


Figure 4.1.1: System Architecture

1. **Cloud Storage Provider (CSP):** A semi-trusted entity with significant storage and computational resources. The CSP is responsible for storing the data blocks and their corresponding file tags. It also generates proofs of storage in response to auditing challenges and executes the technical updates for file tags during ownership modifications or transfers.
2. **Third Party Auditor (TPA):** An independent and trusted entity equipped with the expertise and capabilities to perform public auditing. The TPA issues challenges to the CSP and verifies the returned proofs to ensure data integrity on behalf of the group owners, providing a reliable and convincing auditing result.
3. **Sale Group (SG):** An ad-hoc group of data owners who currently share ownership of the data stored in the cloud. The architecture supports dynamic membership within the SG, allowing members to be added or revoked. When the group decides to sell or move its data, it initiates the ownership transfer process.

4. **Buy Group (BG):** This group has the same dynamic attributes as the SG but acts as the recipient of the data ownership. Upon receiving the necessary dynamic tokens from the SG, the BG coordinates with the CSP to update the file tags, thereby assuming full ownership and auditing rights.
5. **Integrity Auditing:** The TPA selects random blocks to challenge the CSP, which then uses the stored BLS-based file tags to generate a compact proof.
6. **Ownership Modification:** When owners are added or revoked, the group generates an Ownership Dynamic Token (ODT), allowing the CSP to update the tags without requiring the remaining owners to re-calculate everything from scratch.
7. **Ownership Transfer:** Ownership is transferred from the SG to the BG through an interactive process where secure tokens are passed from the selling group to the buying group and finally to the CSP to re-compute the tags.

4.1.2 Module Design

Cloud-Based Auditing System with Efficient Ownership Management for Group Data Transaction is divided into several functional modules. Each module is responsible for performing a specific task and works together with other modules to provide secure cloud storage, efficient auditing, and dynamic ownership management. The modular architecture improves system scalability, maintainability, security, and performance.

Module 1: User Management Module

Purpose: The User Management Module is responsible for handling user registration, authentication, authorization, and account management. It ensures that only authorized users can access cloud resources and perform ownership-related operations.

- **Inputs:** User registration details; login credentials; user profile information
- **Outputs:** User account creation confirmation; authentication status; access permissions
- **Functionalities:** User registration and account creation; secure login and logout; password management; user profile maintenance; role-based access control

Module 2: Cloud Data Storage Module

Purpose: This module manages secure storage of files and data in the cloud environment. It ensures efficient storage, retrieval, and maintenance of cloud data while preserving integrity and availability.

- **Inputs:** User-uploaded files; metadata information; storage requests
- **Outputs:** Stored cloud data; storage location details; file access information
- **Functionalities:** Data upload and storage; data retrieval and download; cloud file management; metadata generation; storage optimization; data backup support

Module 3: Data Integrity Auditing Module

Purpose: The auditing module verifies that cloud-stored data remains intact and unmodified. It allows users and auditors to validate data integrity without downloading the complete file.

- **Inputs:** Stored data blocks; authentication tags; audit requests
- **Outputs:** Integrity verification results; audit reports; data status information
- **Functionalities:** Integrity verification; challenge-response auditing; block verification; batch auditing; detection of data modification; audit log maintenance

Module 4: Third-Party Auditor (TPA) Module

Purpose: This module enables independent auditing of cloud data by a Third-Party Auditor without revealing the actual content of stored files.

- **Inputs:** Audit requests; verification parameters; authentication information
- **Outputs:** Verification reports; audit status; integrity confirmation
- **Functionalities:** Public auditing; batch verification; audit report generation; verification of ownership records; security monitoring; compliance checking

Module 5: Ownership Management Module

Purpose: The Ownership Management Module manages ownership information associated with cloud data. It maintains ownership records and supports dynamic owner management.

- **Inputs:** Ownership requests; user information
- **Outputs:** Updated ownership records; ownership validation results
- **Functionalities:** Owner registration; owner addition; owner revocation; ownership modification; ownership verification; ownership history tracking

Module 6: Ownership Transfer Module

Purpose: This module enables secure transfer of ownership between individuals or groups while maintaining data integrity and security.

- **Inputs:** Ownership transfer requests; existing ownership information; new owner details
- **Outputs:** Transfer confirmation; updated ownership information; transfer logs
- **Functionalities:** Ownership transfer processing; group-to-group ownership transfer; partial ownership transfer; ownership token generation; transfer validation; secure ownership updates

Module 7: Security and Cryptographic Module

Purpose: This module provides the security foundation of the system through cryptographic algorithms and secure authentication mechanisms.

- **Inputs:** User credentials; data blocks; cryptographic keys
- **Outputs:** Digital signatures; authentication tags; encrypted information

- **Functionalities:** BLS Aggregate Signature generation; key generation and management; data authentication; random masking implementation; user authorization; protection against collusion attacks; protection against rogue-key attacks

Module 8: Reporting and Monitoring Module
Purpose: This module continuously monitors system activities and generates reports related to auditing, ownership transactions, and security events.

- **Inputs:** Audit logs; ownership records; user activities; system events
- **Outputs:** Audit reports; monitoring reports; performance statistics; activity logs
- **Functionalities:** Audit report generation; ownership transaction tracking; user activity monitoring; security event monitoring; performance analysis; system status reporting

Overall Module Workflow

1. Users register and authenticate through the User Management Module.
2. Data is uploaded and stored securely using the Cloud Data Storage Module.
3. Authentication tags and signatures are generated by the Security Module.
4. The Auditing Module periodically verifies data integrity.
5. The Third-Party Auditor performs independent public auditing.
6. Ownership records are maintained through the Ownership Management Module.
7. Ownership transfers are securely processed through the Ownership Transfer Module.
8. All auditing and ownership activities are tracked by the Reporting and Monitoring Module.

4.2 Data Flow Diagram

4.2.1 Data Flow Diagram (Level 0)

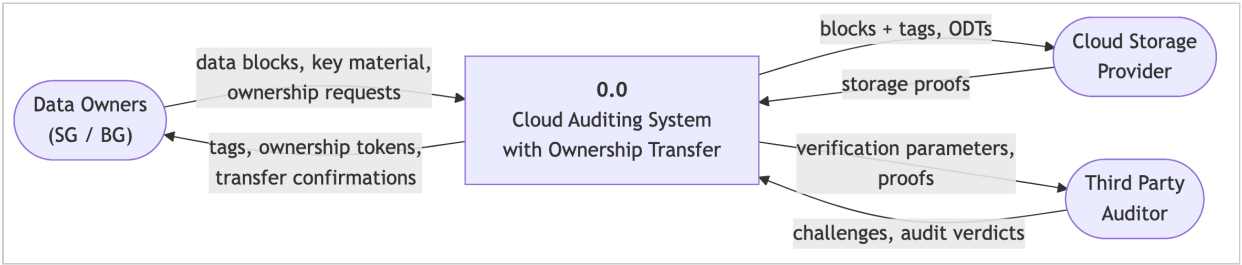


Figure 4.2.1: Data Flow Diagram (Level 0)

4.2.2 Data Flow Diagram (Level 1 and Level 2)

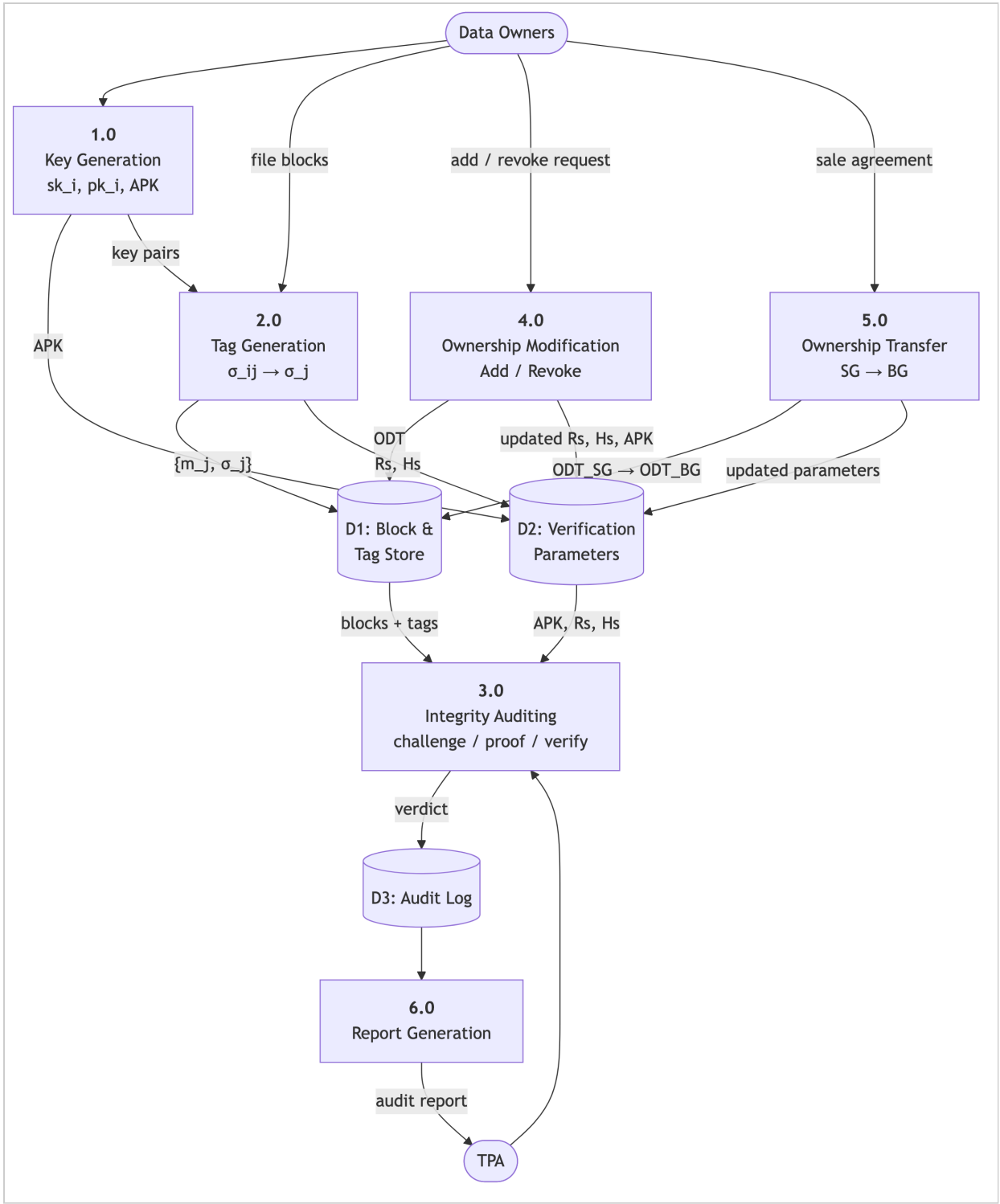


Figure 4.2.2: Data Flow Diagram (Level 1)

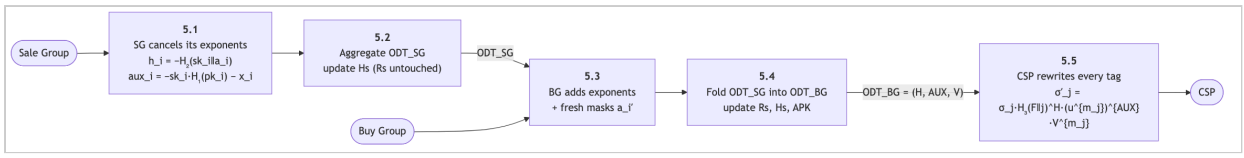


Figure 4.2.2(b): Data Flow Diagram (Level 2 — Ownership Transfer)

4.3 Low Level Design

The Level 0 Data Flow Diagram (DFD) provides a high-level overview of the system and illustrates the interaction between external entities and the main process. The Data Owner/Group Owner registers, uploads files, creates groups, manages ownership, and initiates ownership transfer requests. Users/Group Members access shared data, submit file access requests, and receive system responses. The Cloud Storage Server stores and retrieves data as required by the system. The Third-Party Auditor (TPA) performs independent verification of data integrity and generates audit reports. The Key Generation Center (KGC) generates and distributes public and private cryptographic keys required for secure operations. All requests from external entities are processed by the central system, which manages data storage, auditing, ownership control, and security functions. The processed results, audit reports, and confirmations are then returned to the respective entities. Thus, the Level 0 DFD represents the overall flow of information between the system and its external components.

4.3.1 Class Diagram

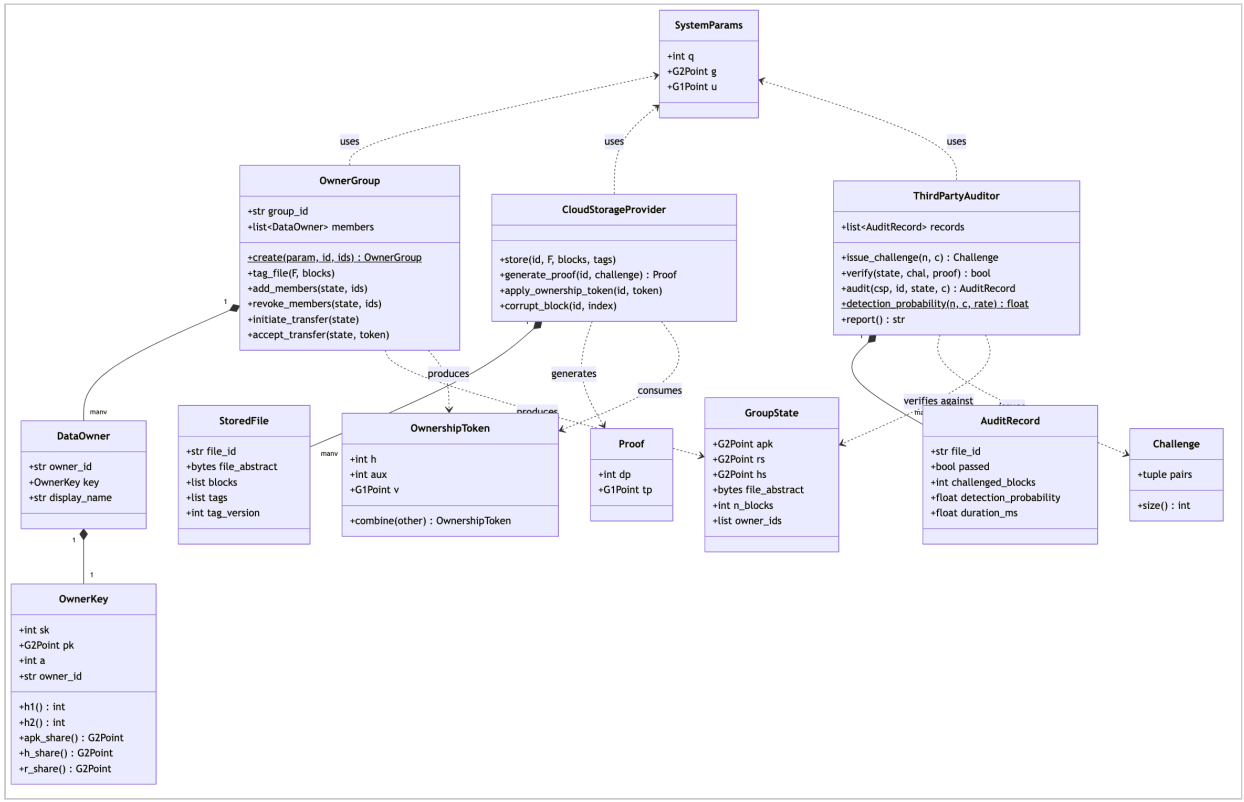


Figure 4.3.1: Class Diagram

The Class Diagram represents the static structure of the system by illustrating the classes, their attributes, methods, and relationships. The User class manages user activities such as registration, login, logout, and profile updates, while the Group class maintains group information and member management. The File class handles file details and supports upload and download operations, whereas the CloudStorage class manages file storage, retrieval, deletion, and availability in the cloud. The Ownership class maintains file ownership information and supports ownership verification, addition, and revocation. The OwnershipTransfer class facilitates secure ownership transfer between users by processing transfer requests and approvals. The Audit class records auditing activities and integrity verification results, while the ThirdPartyAuditor class independently performs audits, generates challenges, verifies proofs, and creates audit reports. The KeyManager class manages cryptographic keys for secure communication and authentication. Finally, the AuditReport class stores the auditing outcomes, including verification status, report details, and remarks generated after the audit process.

4.3.2 UML Diagram

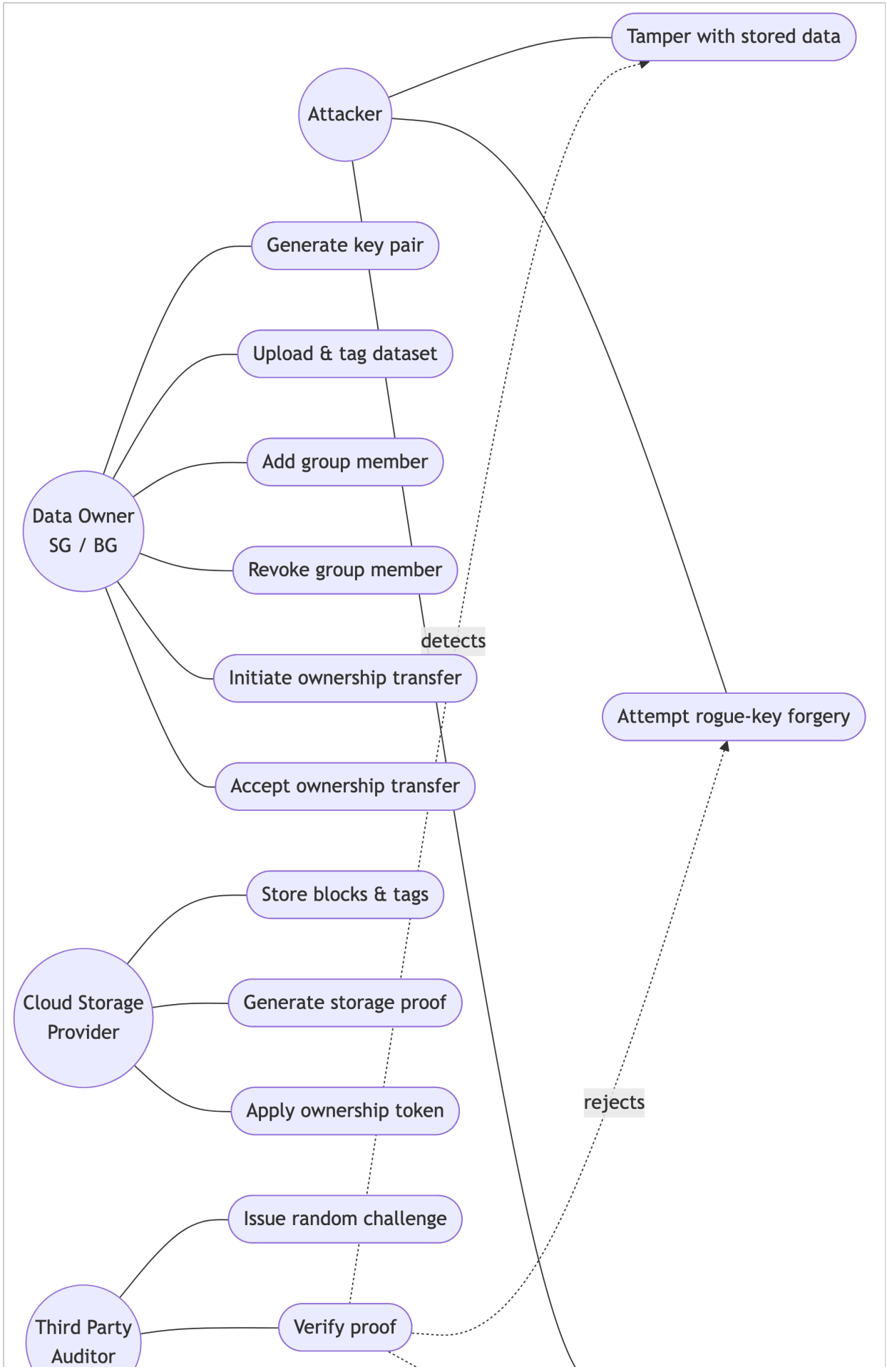

Tamper with stored data
Attacker
Generate key pair
Upload & tag dataset
Add group member
Data Owner
SG / BG
Revoke group member
detects
Initiate ownership transfer
Attempt rogue-key forgery
Accept ownership transfer
Store blocks & tags
Cloud Storage
Provider
Generate storage proof
Apply ownership token
rejects
Issue random challenge
Third Party
Auditor
Verify proof

Figure 4.3.2: UML Diagram (Use Case Diagram)

4.3.3 Activity and Sequence Diagram

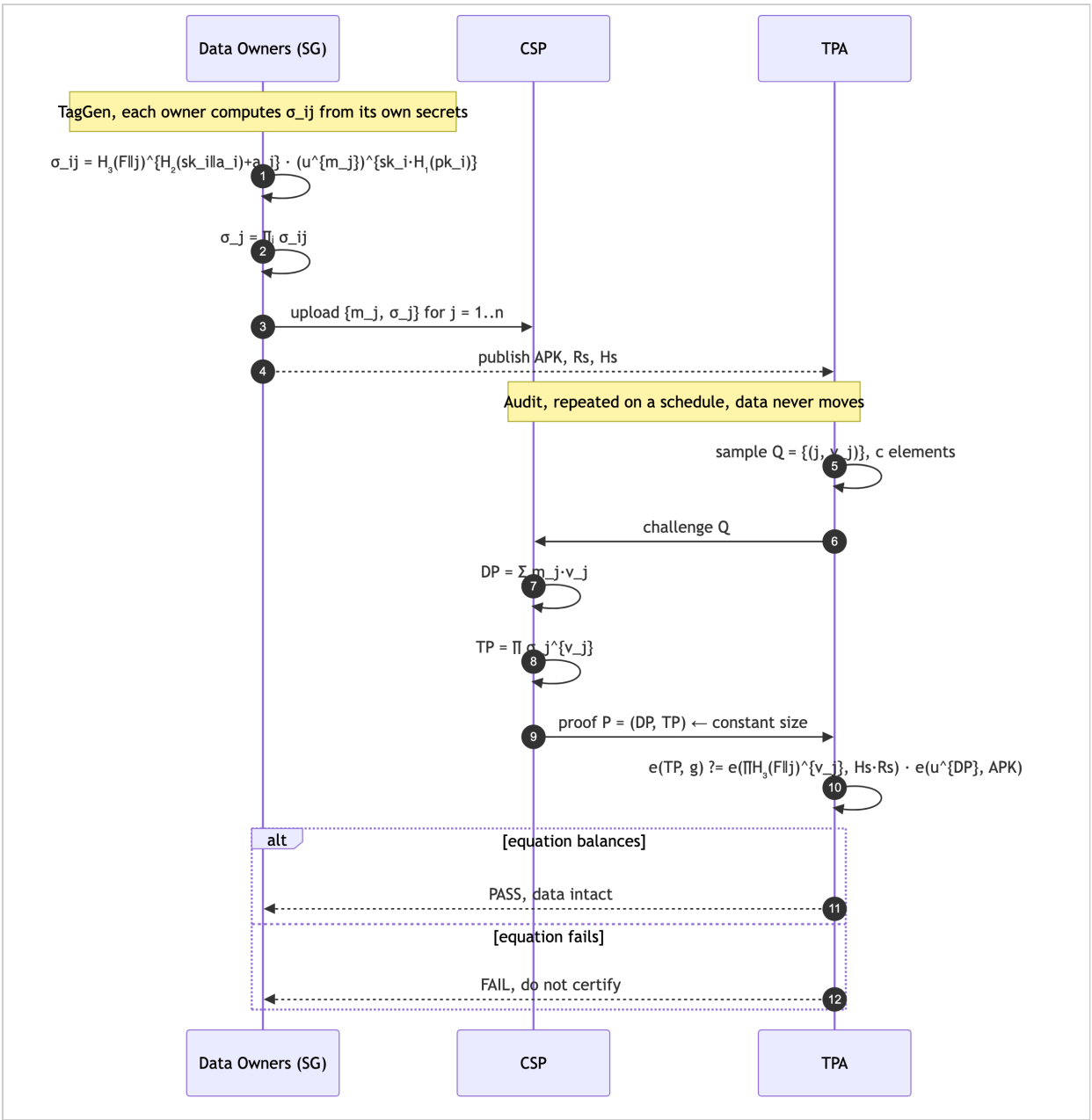


Figure 4.3.3(a): Sequence Diagram — Integrity Auditing

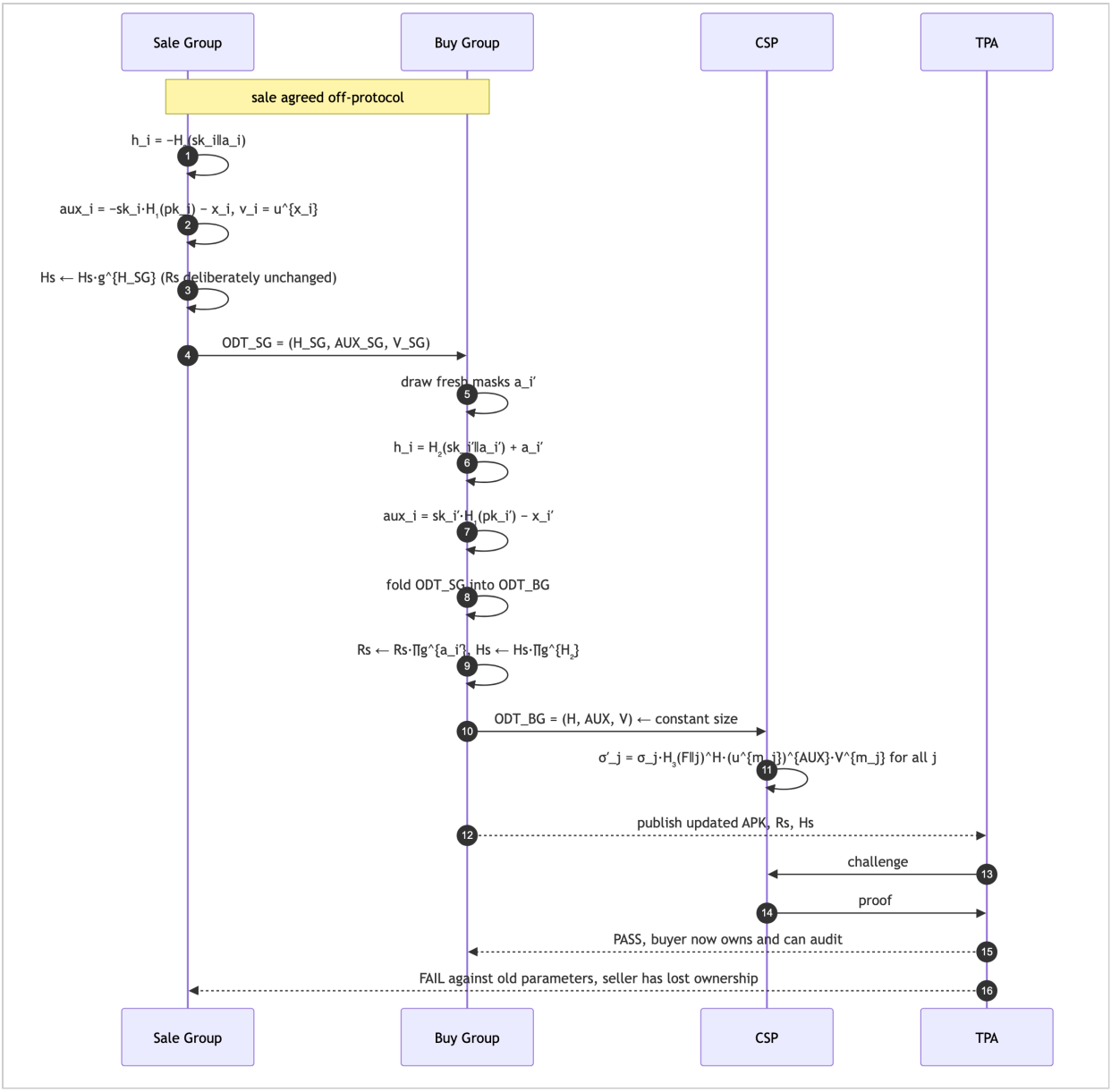
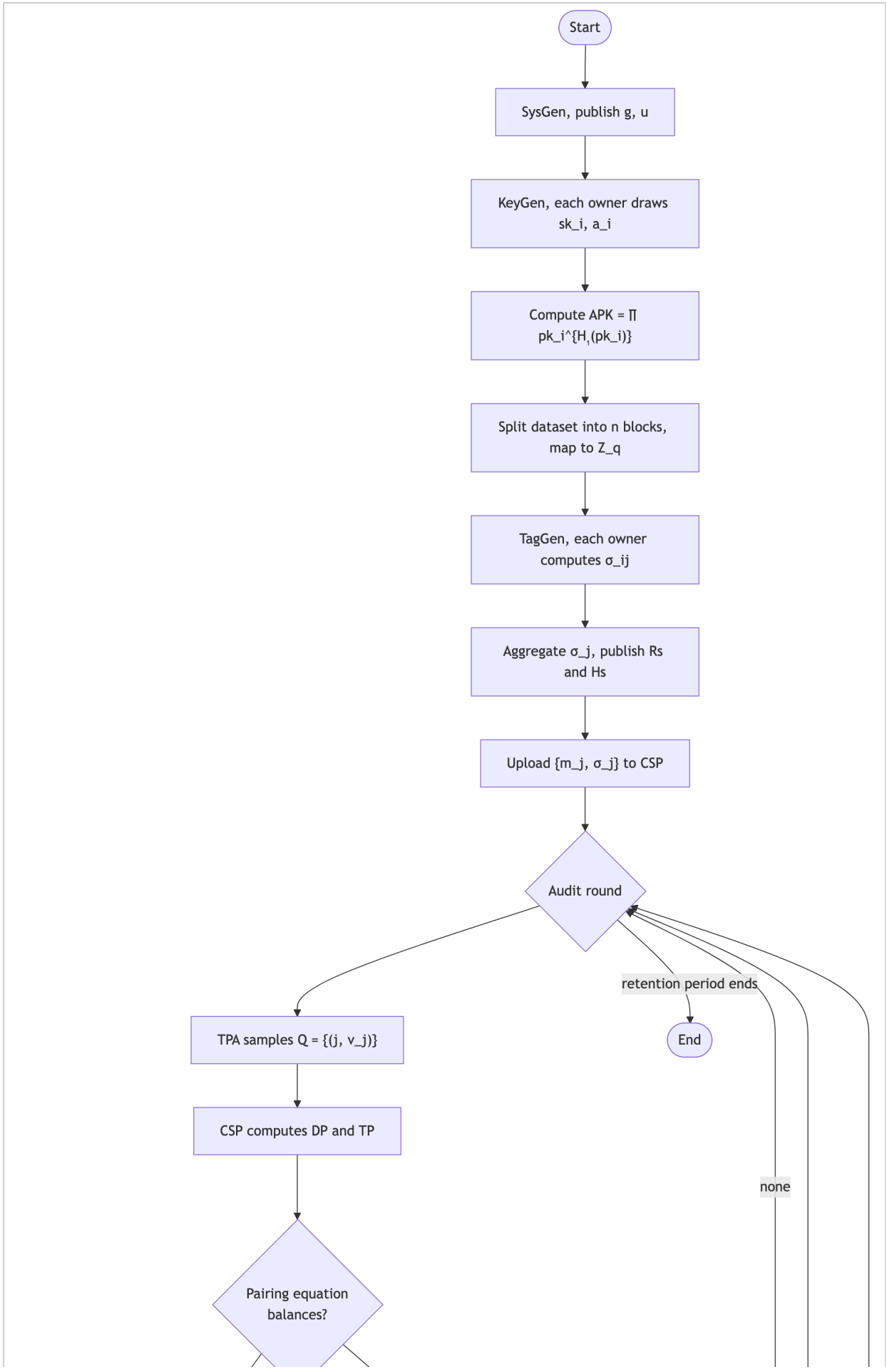


Figure 4.3.3(b): Sequence Diagram — Ownership Transfer


Start
SysGen, publish g, u
KeyGen, each owner draws sk_i, a_i
Compute APK = ∏ pk_i^{H1(pk_i)}
Split dataset into n blocks, map to Z_q
TagGen, each owner computes σ_ij
Aggregate σ_j, publish Rs and Hs
Upload {m_j, σ_j} to CSP
Audit round
retention period ends
End
TPA samples Q = {(j, v_j)}
CSP computes DP and TP
none
Pairing equation balances?

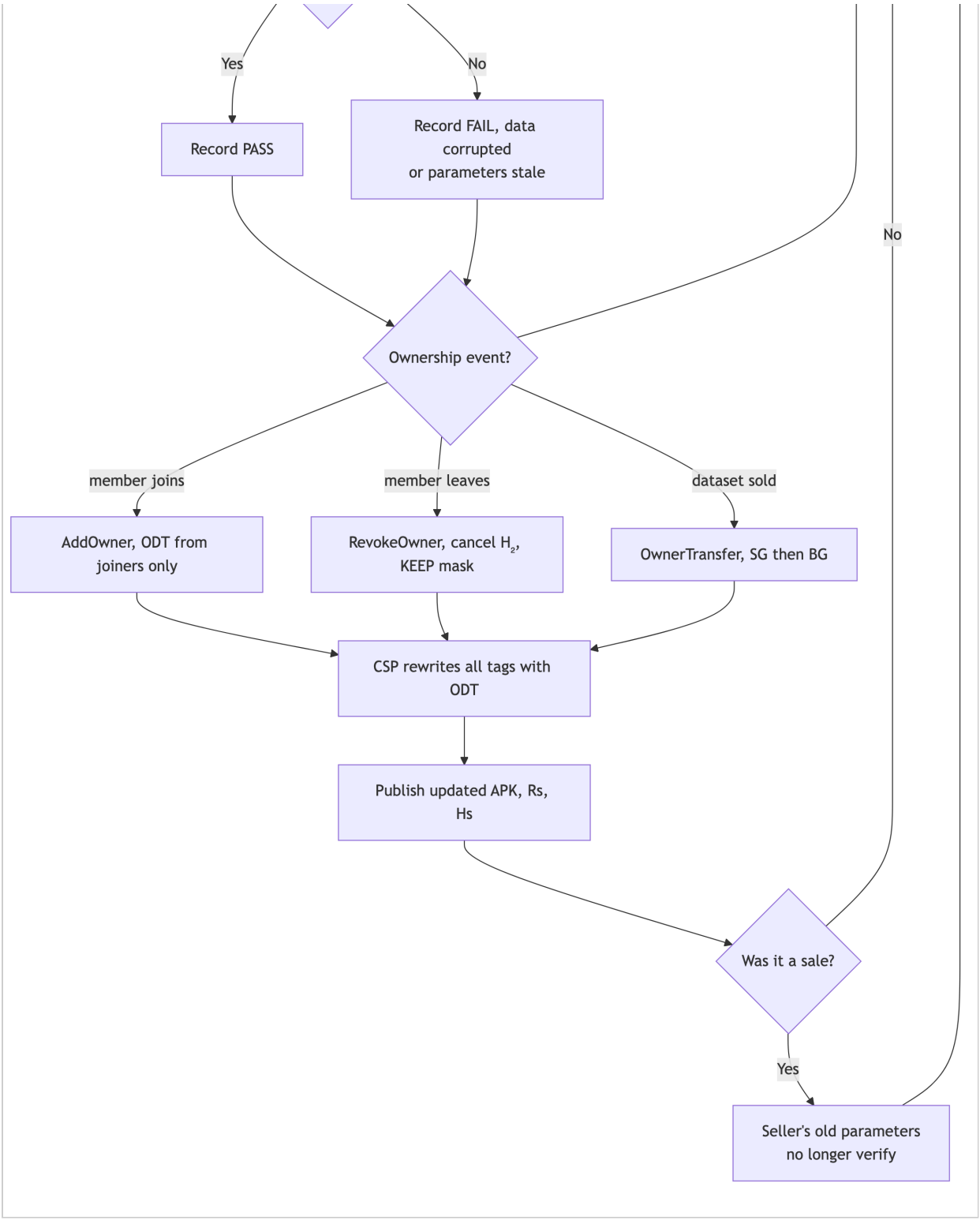


Figure 4.3.3(c): Activity Diagram — Data Lifecycle

4.3.4 Algorithms

Algorithm 1: User Authentication

Steps

1. Start

2. Enter username and password.
3. Validate user credentials.
4. If credentials are correct: grant access; open user dashboard.
5. Else: display “Invalid Username or Password”.
6. End

Pseudocode

```
BEGIN
INPUT username, password
IF credentials are valid THEN
    Grant Access
ELSE
    Display Invalid Login
ENDIF
END
```

Algorithm 2: File Upload

Steps

1. Start
2. User selects file.
3. Validate file format and size.
4. Generate file ID.
5. Create metadata and signature.
6. Store file in cloud server.
7. Save file details in database.
8. Display upload success message.
9. End

Pseudocode

```
BEGIN
SELECT File
VALIDATE File
GENERATE File_ID
GENERATE Metadata
GENERATE Signature
STORE File in Cloud
SAVE Details in Database
DISPLAY Success Message
END
```

Algorithm 3: Data Integrity Auditing

Steps

1. Start
2. Auditor sends challenge request.
3. Cloud server retrieves requested blocks.
4. Generate proof.
5. Verify proof.
6. If proof is valid: integrity maintained.
7. Else: integrity compromised.
8. Generate audit report.
9. End

Pseudocode

```
BEGIN
SEND Challenge
RETRIEVE Data Blocks
GENERATE Proof
VERIFY Proof
IF Proof = Valid THEN
    Integrity Maintained
ELSE
    Integrity Failed
ENDIF
GENERATE Report
END
```

Algorithm 4: Ownership Management

Steps

1. Start
2. Receive ownership request.
3. Verify user authority.
4. Update ownership details.
5. Store updated records.
6. Send confirmation.
7. End

Pseudocode

```
BEGIN
RECEIVE Request
VERIFY User Authority
UPDATE Ownership Record
SAVE Changes
SEND Confirmation
END
```

Algorithm 5: Ownership Transfer

Steps

1. Start
2. Receive ownership transfer request.
3. Verify current owner.
4. Verify new owner.
5. Generate transfer token.
6. Update ownership records.
7. Notify both owners.
8. Complete transfer.
9. End

Pseudocode

```
BEGIN
RECEIVE Transfer Request
VERIFY Current Owner
VERIFY New Owner
GENERATE Transfer Token
UPDATE Ownership Database
NOTIFY Users
DISPLAY Transfer Success
END
```

Algorithm 6: Third Party Auditing

Steps

1. Start
2. Receive audit request.
3. Generate challenge.
4. Cloud server generates proof.
5. Verify proof.

6. Generate audit result.
7. Store audit log.
8. End

Pseudocode

```
BEGIN
RECEIVE Audit Request
GENERATE Challenge
GENERATE Proof
VERIFY Proof
GENERATE Audit Result
STORE Audit Log
END
```

Algorithm 7: BLS Aggregate Signature

Steps

1. Start
2. Generate public and private keys.
3. Create signatures for data blocks.
4. Aggregate signatures.
5. Store aggregate signature.
6. Verify during auditing.
7. End

Pseudocode

```
BEGIN
GENERATE Keys
CREATE Signatures
AGGREGATE Signatures
STORE Aggregate Signature
VERIFY Signature
END
```

Algorithm 8: Report Generation

Steps

1. Start
2. Collect audit records.
3. Collect ownership transaction records.
4. Analyze information.

5. Generate report.
6. Export report.
7. End

Pseudocode

```
BEGIN
COLLECT Records
ANALYZE Data
GENERATE Report
EXPORT Report
END
```

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

The implementation phase translates the design of Chapter 4 into working software. The objective was not to produce a demonstration that merely *looks* like cloud auditing, but to implement the cryptographic protocol of the base paper faithfully enough that its security claims can be tested by running code.

The system implements all six algorithms of the base paper, `SysGen`, `KeyGen`, `TagGen`, `Audit`, `OwnerModify` (comprising `AddOwner` and `RevokeOwner`) and `OwnerTransfer`, together with the four entities of its system model, five large sample datasets, a benchmark suite and three executable attack demonstrations.

5.2 Implementation Environment

Item	Detail
Programming language	Python 3.12
Cryptographic library	`py_ecc` 7.0 (BLS12-381 pairings)
Plotting	`matplotlib` 3.11
Testing framework	`pytest` 7.4
Version control	Git
Development machine	Apple silicon (arm64), macOS
Total dependencies	3

Python was chosen from the two languages permitted by the requirements in §3.2. The dependency count is deliberately small: the protocol itself is implemented from first principles, and no library performs any part of the scheme on the project's behalf.

5.2.1 Selection of the cryptographic library

Three candidate libraries were evaluated on the target machine before selecting one.

Library	Outcome
`blspy` (Chia)	Installs successfully, but exposes **no scalar multiplication** on G_1 or G_2 (`G1Element.__mul__` does not exist). It is a signature-scheme API, not a general pairing library, and therefore cannot implement a custom scheme. **Rejected.**
`petrelic` (RELIC bindings)	No distribution available for arm64 macOS. **Rejected.**
`py_ecc`	Pure Python; installs with no compiler and no system libraries; exposes generic group arithmetic and a raw pairing. **Selected.**

The trade-off is performance, discussed honestly in §7.5.

5.3 Modular Organisation

The codebase is organised so that the protocol can be read on its own, with no framework or storage concerns interleaved.

```
code/
├── crypto/        the protocol, no I/O, no framework, no dependencies
beyond py_ecc
├── entities/      the four actors of the system model, as readable classes
├── datasets/      large synthetic datasets with ownership scenarios
├── demo/          narrated end-to-end walkthrough
├── attacks/       three executable attack demonstrations
├── benchmarks/    measurement harness and figure rendering
└── tests/         41 correctness tests
```

The dependency direction is strictly one-way, `crypto/` does not know that `entities/` exists, and `entities/` does not know that `demo/` exists. A reader can therefore study the cryptography in isolation and find it complete.

5.4 Implementation of the Cryptographic Core

5.4.1 Pairing backend and the Type-1 → Type-3 port

The base paper assumes a **symmetric (Type-1)** bilinear pairing $e: G_1 \times G_1 \to G_T$, as provided by the PBC library's Type-A curves. Type-1 curves are now considered obsolete and are not available in any maintained Python library.

The scheme was therefore ported to **BLS12-381**, an **asymmetric (Type-3)** curve $e: G_1 \times G_2 \to G_T$. The port follows a single placement rule:

> *Every element that is raised to a secret and then published as a verification parameter is placed in **G2**; the tag and its bases are placed in **G1**.*

Element	Group
`u`, `H₃(F‖j)`, tag `σ_j`, proof `TP`	G_1
generator `g`, `pk_i`, `APK`, `Rs`, `Hs`	G_2

Under this placement every pairing in the scheme is a well-formed $G_1 \times G_2$ pairing. The correctness of the audit equation was re-derived algebraically before any code was written, and is reproduced in §5.4.4.

All pairing operations are confined to a single module, `crypto/pairing.py`. No other file imports `py_ecc`. Replacing the backend with a native implementation would therefore be a one-file change.

5.4.2 Correction of the paper's hash-function notation

The base paper is internally inconsistent in its use of hash functions. Section V.B declares

```
H₁ : {0,1}* → G₁        H₂ : {0,1}* → Z_p
```

but then uses `H₁(pk_i)` as an **exponent** (`APK = ∏ pk_i^{H₁(pk_i)}`), which requires `H₁` to map into `Z_q`. It also uses the same value as `H₂(F‖j)` in `TagGen` and as `H₃(F‖j)` in the verification equation and in the correctness proof of Theorem 5.

The implementation uses the assignment the equations actually require, with three distinctly named functions:

Function	Codomain	Purpose
`H₁`	`Z_q`	`H₁(pk_i)`, binds an owner's key pair
`H₂`	`Z_q`	`H₂(sk_i‖a_i)`, the masked index exponent
`H₃`	G_1	`H₃(F‖j)`, hash-to-curve of the block index

This is a correction of notation, not a modification of the scheme: every equation in the paper is reproduced exactly under this reading.

Two implementation details are worth recording. `H₁` and `H₂` reduce a **SHA-512** digest modulo `q`; using SHA-256 against a 255-bit `q` would introduce measurable modular bias, whereas a 512-bit digest reduces the bias below 2^{-255}. `H₃` uses the RFC 9380 SSWU hash-to-curve construction, which is indifferentiable from a random oracle: the assumption under

which Theorems 1–4 of the paper are stated. All three use domain-separation tags and length-prefixed inputs, so `H(a‖b)` is unambiguous.

5.4.3 Independence of the generators

`u` must not be a known power of `g`. If any party knew `log_g(u)`, the unforgeability reduction of Theorem 2 would collapse and that party could forge tags for arbitrary blocks.

The implementation derives `u` by hash-to-curve from a fixed public string, a construction known as *nothing-up-my-sleeve*. Because `u` is the output of a hash rather than a chosen value, no party (including the implementers) can know the discrete logarithm relating it to `g`.

5.4.4 The six algorithms

SysGen. Fixes the curve, the two generators and the hash functions.

KeyGen. Each owner independently draws `sk_i ← Z_q` and publishes `pk_i = g^{sk_i}`, together with a secret random mask `a_i`. There is no dealer, no shared secret and no interaction between owners. The group's aggregated public key is

```
APK = ∏ᵢ pk_i^{H₁(pk_i)}
```

Binding each public key with a hash of itself is what makes the scheme resistant to rogue-key attacks, at a cost of **one exponentiation per owner**, O(s). The baseline requires each owner to construct an authenticator against every other owner, costing O(s²).

TagGen. Each owner computes, for each block,

```
σ_ij = H₃(F‖j)^{H₂(sk_i‖a_i) + a_i} · (u^{m_j})^{sk_i·H₁(pk_i)}
```

and the shares are aggregated into one tag per block, `σ_j = ∏ᵢ σ_ij`. Only the aggregates `Rs = ∏ g^{a_i}` and `Hs = ∏ g^{H₂(sk_i‖a_i)}` are published; the individual masks `a_i` never are.

Audit. The TPA samples a challenge `Q = {(j, v_j)}` of `c` elements; the CSP replies with `DP = Σ m_j v_j` and `TP = ∏ σ_j^{v_j}`; the TPA verifies

```
e(TP, g) = e(∏ H₃(F‖j)^{v_j}, Hs·Rs) · e(u^{DP}, APK)
```

Correctness under the Type-3 port. Writing `A = Σᵢ (H₂(sk_i‖a_i) + a_i)` and `B = Σᵢ sk_i·H₁(pk_i)`, so that `σ_j = H₃(F‖j)^A · (u^{m_j})^B`:

```
TP = ∏ⱼ σ_j^{v_j} = (∏ⱼ H₃(F‖j)^{v_j})^A · u^{B·Σⱼ m_j v_j}
                  = (∏ⱼ H₃(F‖j)^{v_j})^A · u^{B·DP}

e(TP, g) = e(∏ⱼ H₃(F‖j)^{v_j}, g^A) · e(u^{DP}, g^B)
         = e(∏ⱼ H₃(F‖j)^{v_j}, Hs·Rs) · e(u^{DP}, APK)
```

since `Hs·Rs = g^A` and `APK = g^B`. ∎

OwnerModify and OwnerTransfer. Every ownership operation reduces to a constant-size **Ownership Dynamic Token** `ODT = (H, AUX, V)`, with which the CSP rewrites each tag in place:

```
σ'_j = σ_j · H₃(F‖j)^H · (u^{m_j})^{AUX} · V^{m_j}
```

Three properties of the token are deliberate:

1. **Constant size.** `H` and `AUX` are scalars and `V` is one group element, 160 bytes in total, independent of both `n` and `s`. The O(n) rewriting work falls on the CSP, which the system model describes as the party with "abundant storage and computational resources".
2. **Blinding.** Each owner splits its contribution as `aux_i = sk_i·H₁(pk_i) − x_i` with `v_i = u^{x_i}`, so `AUX` alone never reveals the group's aggregate signing exponent. The CSP recombines the parts only inside the exponent of `u`.
3. **Masks are never subtracted.** A joining owner contributes `H₂(sk_i‖a_i) + a_i`; a leaving owner cancels only `H₂(sk_i‖a_i)`, and `Rs` is left unchanged. The departing owner's mask therefore remains embedded in every tag permanently.

The third property is the security-critical one, and it is the reason the scheme resists the collusion attack. It is pinned by a dedicated test (§6.4) so that a future refactoring cannot silently remove it under the impression that it is an inconsistency.

5.5 Implementation of the System Entities

The four entities of the system model are implemented as classes whose boundaries encode the trust model.

Class	Trust level	Holds	Never receives
`CloudStorageProvider`	semi-trusted	blocks, tags, tokens	secret keys, masks, un-blinded exponents
`ThirdPartyAuditor`	independent	challenges, proofs, `APK` / `Rs` / `Hs`	block contents, any secret
`OwnerGroup` (SG and BG)	owners	own `sk_i` and `a_i`	other owners' secrets

`OwnerGroup` serves as both the Sale Group and the Buy Group, because "selling" and "buying" are roles within a single transaction rather than permanent properties: a research institute that buys a dataset today is the sale group when it resells tomorrow.

`CloudStorageProvider` deliberately exposes `corrupt_block()` and `delete_block()`. These are not defects but the modelled attack surface of a semi-trusted cloud, and they are exercised by the demonstrations in Chapter 6.

5.6 Sample Datasets

The base paper motivates itself with a concrete situation: organisations such as Kaggle and Zenodo hold large AI-related datasets and **sell** them to other organisations. Demonstrating that requires datasets that plausibly get sold: large, owned by a consortium rather than an individual, and carrying provenance that must survive the transfer. No public corpus of "datasets with transferable ownership records" exists, so five were generated.

Dataset	Sale Group → Buy Group
`medical-imaging`	3 hospitals → medical-AI research institute
`iot-telemetry`	city transport dept + 2 agencies → urban analytics firm
`financial-ledger`	regional bank → statutory audit firm
`genomics-variants`	3 sequencing labs → pharmaceutical company
`ai-training-corpus`	data-labelling vendor → autonomous-systems AI lab

Available at four size tiers (2 MB, 25 MB, 100 MB and 512 MB) sharded into 8 MB parts in the manner of a real data lake, so that a buyer can audit part of a purchase without retrieving the whole archive. 500 MB was generated for this project.

All data is synthetic. No real clinical, financial or personal record is used or reproduced; the generators emit statistically plausible but entirely fabricated records. This is what a research demonstration requires and it avoids every privacy concern that real data would raise. Generation is deterministic: the same seed reproduces a dataset byte for byte, so audits and benchmarks remain reproducible.

5.6.1 Mapping file blocks into Z_q

A BLS12-381 scalar holds approximately 255 bits, so a real file block cannot be embedded directly. Two modes are provided:

- **`hashed`** (default), `m_j = SHA-512(block) mod q`, for any block size. Soundness is preserved, because computing the hash still requires possessing the block: a cloud that has

discarded the data cannot produce `m_j` and therefore cannot produce a valid `DP`.

- **`raw`**, `m_j = int(block) mod q` for blocks of at most 31 bytes. This is the mode the base paper uses (8-byte blocks) and is used by the benchmark suite for parity.

5.6.2 The significance of block size

Protocol cost is driven by the **number of blocks** `n`, not by file size:

```
tagging   = n · s group exponentiations
storage   = n · 96 bytes of tags
auditing  = c hash-to-curve operations + 3 pairings
```

At the default 256 KB block size, a 100 MB dataset is 400 blocks. Security is unaffected by this choice: the detection bound depends on the *fraction* of blocks corrupted, not on their size, so challenging a fixed number of blocks detects a given corruption rate equally well at any block size. Larger blocks in fact mean that an attacker who corrupts one block has damaged more bytes.

5.7 Summary

The implementation covers all six algorithms of the base paper, ported to a modern pairing curve with the correctness of the audit equation re-derived under that port. Two ambiguities in the paper's notation were identified and resolved. The cryptographic core is isolated from all I/O concerns, and the pairing backend is confined to a single replaceable module.

The next chapter reports the testing performed against this implementation, including the three attack demonstrations that test the paper's security claims directly.

CHAPTER 6

TESTING

6.1 Introduction

Testing a cryptographic protocol differs from testing ordinary application software. The question is never "does this function execute without error": a broken implementation of a signature scheme executes perfectly well and simply accepts forgeries. The question is always **"does the security property still hold after this operation?"**

The test strategy therefore has two halves:

1. **Correctness testing**, 41 automated tests asserting that the audit equation holds when it should, and fails when it should.
2. **Security testing**, three executable attack demonstrations that attempt real forgeries against the implementation, and against the prior-art schemes it replaces.

The second half is the more important. A test suite proves the implementation does what the authors intended; an attack demonstration proves that what they intended was worth doing.

6.2 Test Environment

Item	Detail
Framework	`pytest` 7.4
Test cases	41
Execution time	132 seconds
Result	**41 passed, 0 failed**
Command	`pytest tests/ -v`

Small parameters are used throughout (6 blocks, 1–5 owners). The tests verify **algebra**, not performance, and larger parameters would only make the suite slower without testing anything further.

6.3 Correctness Testing

6.3.1 Test coverage by module

Test class	Cases	What it establishes
`TestSysGenKeyGen`	4	Generators are deterministic and independent; keys and masks are distinct per owner; `APK` is order-independent
`TestHashes`	4	H_1, H_2 land in Z_q; H_3 is deterministic, index-sensitive and file-sensitive
`TestTagGenAudit`	9	Honest audits pass; corruption, deletion, block substitution, forged tags and wrong file abstracts all fail
`TestAddOwner`	4	Auditing survives owner addition; stale parameters are rejected; operations compose
`TestRevokeOwner`	4	Auditing survives revocation; **masks are provably retained**
`TestOwnerTransfer`	5	The buyer gains auditing ability, the seller loses it; chained resales work; the token is constant-size
`TestSerialisation`	6	Round-trips are exact and canonical; off-curve points are rejected
`TestEndToEnd`	1	Complete lifecycle, all six algorithms in sequence

6.3.2 Representative negative tests

The negative tests are the substantive ones: they assert that the protocol *refuses* things it should refuse.

Test	Attack modelled	Expected
`test_corrupted_block_fails`	Cloud alters one block	Audit fails
`test_deleted_block_fails`	Cloud loses a block and answers with zeros	Audit fails
`test_swapped_blocks_fail`	Cloud serves block 4's contents for a challenge on block 1	Audit fails
`test_forged_tag_fails`	Tag invented using an unrelated key	Audit fails
`test_wrong_file_abstract_fails`	Tags from one file replayed for another	Audit fails
`test_stale_state_rejected_after_add`	Old verification parameters used after a membership change	Audit fails

Test	Attack modelled	Expected
`test_seller_loses_ability_to_audit`	Seller attempts to audit after a sale	Audit fails

All behave as expected.

6.3.3 A defect found and fixed during testing

Three tests failed on their first run. The cause was not in the protocol but in how group elements were compared.

`py_ecc` represents curve points in **projective** coordinates `(x, y, z)`, and a single curve point has infinitely many such representations, `(x, y, 1)` and `(4x, 8y, 2)` denote the same point. Comparing the raw tuples with `==` therefore compares *representations*, not *points*.

The failure was subtle in an instructive way: the comparison happens to succeed whenever both operands were produced by identical sequences of operations, so the bug is intermittent and easy to miss. It surfaced in `test_apk_is_order_independent`, where the same aggregate public key was computed in two different orders.

The fix was made in the library layer rather than in the tests, `g1_eq()` and `g2_eq()` were added to `crypto/pairing.py`, normalising to affine coordinates before comparison, because any future code comparing group elements would hit the same trap.

Two further tests were strengthened as a result. `test_masks_are_retained_on_revoke` had been comparing object identity and therefore passing for the wrong reason; it now asserts the exact algebra, namely that `Hs` shrinks by precisely `g^{H₂(sk‖a)}` of the revoked owner. A new test, `test_encoding_is_canonical`, confirms that two projective representations of the same point serialise to identical bytes.

6.3.4 The security-critical test

```
tests/test_protocol.py::TestRevokeOwner::test_masks_are_retained_on_revoke
```

This test asserts that `Rs` is **unchanged** by revocation, while `Hs` shrinks by exactly the revoked owner's `H₂` term.

To a reader unfamiliar with the scheme, retaining a departed owner's random mask looks like an oversight, and "tidying it up" would appear to be a harmless simplification. It is not: removing it would silently destroy the defence against the collusion attack while leaving every other test passing and the system apparently working. The test exists to make that impossible.

6.4 Security Testing: Attack Demonstrations

Each demonstration executes the same attack twice: once against the prior-art scheme it targets, and once against the scheme implemented here. Demonstrating only the defence would prove nothing, because a scheme that rejects everything also rejects attacks.

6.4.1 Attack 1: Collusion between the buyer and the cloud

Command: `python -m attacks.collusion`

This is the vulnerability the base paper exists to fix (section III.D).

The setting. In schemes [7] and [8], completing a sale requires the seller to hand its index secret key sk_1 to the buyer in the clear. The buyer then colludes with the cloud. Between them they hold the leaked key, every stored tag and every stored block.

Why the forgery works. The prior-art tag separates cleanly into a position factor and a contents factor:

$$\sigma_j = H_0(j)^{sk_1} \cdot (u^{m_j})^{sk_2}$$

The position factor depends only on j and sk_1. Holding sk_1, an attacker can peel it off one tag and graft it onto another:

$$\sigma' = \sigma_{j'} \cdot H_0(j')^{-sk_1} \cdot H_0(j)^{sk_1} = H_0(j)^{sk_1} \cdot (u^{m_{j'}})^{sk_2}$$

That is a valid tag binding position j to the contents of block j'. A cloud that has **deleted** block j can therefore answer a challenge for j using any block it still holds. The entire attack costs two exponentiations and one multiplication.

Result.

Target	Outcome
Prior art [7], [8]	**BROKEN**, the forged proof for a deleted block was accepted
Proposed scheme	**RESISTED**, the identical forgery was rejected

The demonstration deliberately grants the attacker *more* than the protocol would ever leak: every owner's $H_2(sk_i \| a_i)$ value is handed over. The attack still fails, because the graft removes only the H_2 component and leaves the random masks a_i welded to the original index. Recovering a_i from the published $Rs = \prod g^{a_i}$ would require solving a discrete logarithm.

A follow-up check confirms that honest proofs still verify afterwards, so the defence causes no collateral damage.

6.4.2 Attack 2: Rogue-key attack on aggregated ownership

Command: `python -m attacks.rogue_key`

The setting. Under naive BLS aggregation, `APK = ∏ pk_i`. An attacker who chooses its key *after* seeing an honest party's key can publish

```
pk_rogue = g^α · pk_honest^{-1}
```

so the aggregate collapses to `g^α`, which it fully controls. It then signs alone and the verification equation accepts, "proving" that the honest party co-signed a message it has never seen. In this system that means claiming co-ownership of another group's dataset.

The defence. Binding each key with a hash of itself, `APK = ∏ pk_i^{H₁(pk_i)}`, means a valid signature would require the exponent

```
sk·H₁(pk_honest) + sk_rogue·H₁(pk_rogue)
```

Substituting the attacker's own construction `sk_rogue = α − sk` gives

```
sk·(H₁(pk_honest) − H₁(pk_rogue)) + α·H₁(pk_rogue)
```

Because the two hash values differ, the honest secret `sk` no longer cancels, and the attacker does not know `sk`.

Result.

Target	Outcome
Plain aggregate BLS	**BROKEN**, fake co-ownership accepted
Proposed `APK`	**RESISTED**, both the attacker's best attempt and the plain-aggregation forgery rejected
Genuine joint signature	**Still verifies**, the defence does not break the legitimate case

The cost of this defence is one extra exponentiation per owner, O(s). The baseline achieves the same protection with a public-key-aggregation authenticator costing $O(s^2)$, which must additionally be rebuilt on every membership change.

6.4.3 Attack 3: Data tampering by a semi-trusted cloud

Command: `python -m attacks.tampering`

Deterministic detection. When a corrupted block is challenged, the audit fails without exception: corrupting `m_j` breaks the homomorphic relation between `DP` and `TP`, and the pairing equation cannot balance. Both a single altered block and a deleted block answered with zeros were detected.

Probabilistic coverage. The auditor samples `c` blocks rather than all `n`, so the practical question is whether the sample hits a corrupted block. The base paper uses the classical PDP bound

```
P_detect = 1 − ((n − t)/n)^c
```

to justify `c = 460`.

A finding. This bound models sampling **with replacement**. The implementation's auditor draws **distinct** block indices, for which the exact model is hypergeometric:

```
P_detect = 1 − C(n−t, c) / C(n, c)
```

Measured detection rates over 500 trials per configuration track the hypergeometric model closely and sit **at or above** the paper's bound in every configuration tested. Detailed figures appear in §7.4.

The conclusion is favourable to the paper: its bound is *conservative*, never optimistic, so the recommended `c = 460` is safe. It under-promises what the implementation actually delivers.

6.5 System Testing: End-to-End Walkthrough

Command: `python -m demo.run_demo`

A narrated walkthrough runs the complete lifecycle against a real generated dataset, checking the auditor's verdict at each stage.

#	Step	Expected	Result
1	Honest cloud audited	PASS	PASS
2	Cloud corrupts a block	FAIL	FAIL
3	Block restored	PASS	PASS
4	New institution joins the consortium	PASS	PASS
5	An institution leaves	PASS	PASS
6	Dataset sold, **buyer** audits	PASS	PASS
7	Dataset sold, **seller's old parameters**	FAIL	FAIL

All 7 checks behaved as expected.

Step 7 is the defining result of the project. After the sale, the seller's verification parameters are worthless against the re-tagged data. Ownership did not change merely in a database record: it changed cryptographically, which is precisely the property that ordinary access-control systems cannot provide.

6.6 Summary of Test Results

Category	Tests	Result
Correctness (`pytest`)	41	**41 passed**
Collusion attack	prior art vs proposed	Prior art broken, proposed resisted
Rogue-key attack	plain BLS vs proposed	Plain BLS broken, proposed resisted
Tampering detection	deterministic + 3,500 sampling trials	All corruptions detected; bound confirmed conservative
End-to-end lifecycle	7 checks	**7 as expected**

One implementation defect was found and fixed during testing (§6.3.3), in the point-comparison layer rather than the protocol. No defect was found in the protocol implementation itself.

CHAPTER 7

RESULTS AND PERFORMANCE ANALYSIS

7.1 Introduction

This chapter reports the measured performance of the implemented system, reproducing the experimental programme of Figs. 5–8 of the base paper. The purpose is to test the paper's central performance claim:

> *Key generation and group membership changes cost **O(s)** in the proposed scheme, versus **O(s²)** in the baseline (scheme [8]), where `s` is the number of owners.*

An asymptotic claim is established by the **shape** of a curve, not by absolute milliseconds. Accordingly, both schemes are measured on the same curve (BLS12-381), through the same pairing backend, in the same process, on the same machine, so that the comparison is fair even though the absolute figures differ greatly from the paper's C implementation (§7.6).

7.2 Experimental Setup

Item	Detail
Machine	Apple silicon (arm64), macOS
Python	3.12.3
Pairing backend	`py_ecc` (pure Python), BLS12-381
Security level	128-bit
Repeats	3 per configuration; **median** reported
Total runtime	39.3 minutes
Block mode	`raw` (8-byte blocks), matching the paper

The median rather than the mean is reported: a single garbage-collection pause or operating-system scheduling event would distort a mean, and typical cost is what matters.

7.2.1 Primitive costs

All results below are ultimately composed of these four operations:

Operation	Time
Hash-to-curve into G_1	4.8 ms
G_1 scalar multiplication	10.3 ms
G_2 scalar multiplication	37.7 ms
Bilinear pairing	578 ms

Because a pairing costs roughly 56× a G_1 multiplication, the implementation batches the audit's three pairings into a single final exponentiation, reducing verification from approximately 1.7 s to 0.4 s.

7.2.2 Validation of measurement quality

Two checks were performed on the data before it was used.

Contention check. Other processes ran on the machine during part of the benchmark, which could inflate timings. A clean re-run of the `KeyGen` section, with nothing else executing, was compared against the recorded values:

`s`	Series	Recorded	Clean re-run	Deviation
4	proposed	298 ms	297 ms	−0.3 %
10	proposed	746 ms	742 ms	−0.5 %
20	proposed	1,486 ms	1,476 ms	−0.6 %
4	baseline	745 ms	733 ms	−1.6 %
16	baseline	10,660 ms	10,006 ms	−6.1 %
20	baseline	15,540 ms	15,499 ms	−0.3 %

Worst-case deviation 6.1 %, typically under 1 %. The recorded data is therefore sound.

Outlier check. Two points in the original `TagGen` section (`s=1, n=50` and `s=5, n=50`) were **non-monotonic in n**: the `s=1, n=50` figure read 7,858 ms, higher than the same configuration at `n=200` (7,168 ms). This is physically impossible for an O(n) operation and was a measurement artefact. That section was re-measured in a clean run and the corrected values are used throughout. The anomaly is recorded in the results file rather than silently overwritten.

7.3 Key Generation: the headline result

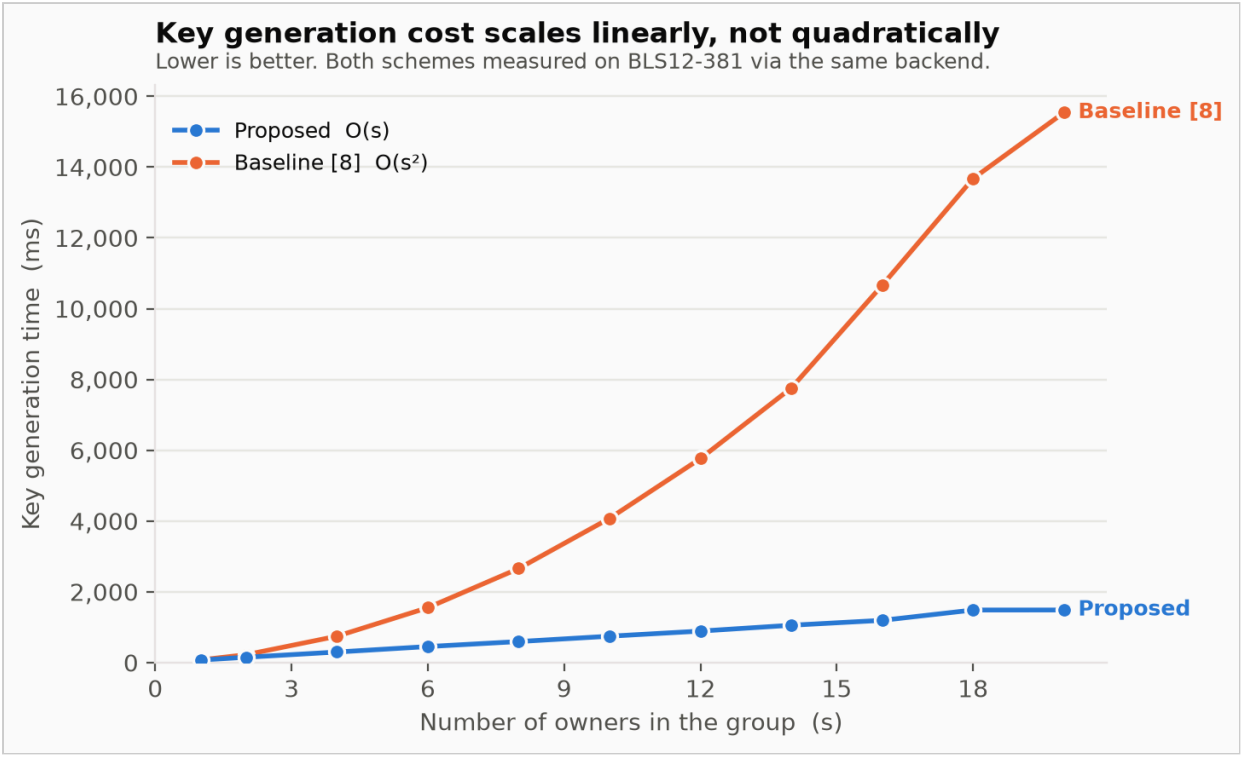


Figure 7.1: Key generation time versus number of owners — proposed O(s) scheme against the O(s²) baseline (reproduces base paper Fig. 6)

Owners `s`	Proposed (ms)	Baseline [8] (ms)	Speedup
1	73	77	1.1×
2	150	223	1.5×
4	298	745	2.5×
6	453	1,552	3.4×
8	594	2,658	4.5×
10	746	4,076	5.5×
12	890	5,765	6.5×
14	1,055	7,758	7.4×
16	1,193	10,660	8.9×
18	1,485	13,675	9.2×
20	**1,486**	**15,540**	**10.5×**

7.3.1 Analysis

The clearest evidence is the behaviour when the group **doubles** from 10 to 20 owners:

	at s = 10	at s = 20	ratio	expected
Proposed	746 ms	1,486 ms	**1.99×**	2× for O(s)
Baseline	4,076 ms	15,540 ms	**3.81×**	4× for O(s²)

The proposed scheme doubles; the baseline very nearly quadruples. This is the asymptotic claim confirmed directly.

Equivalently, the **speedup itself grows linearly** with `s`: from 1.1× at one owner to 10.5× at twenty. A ratio that grows linearly is precisely the signature of a quadratic cost divided by a linear one. Extrapolating, a 100-owner consortium would see a speedup of roughly 50×.

The mechanism is straightforward. The baseline requires every owner to construct an authenticator binding it to every other owner, `s` exponentiations each, `s²` for the group. The proposed scheme replaces this with `APK = ∏ pk_i^{H₁(pk_i)}`, in which each owner's exponent depends only on its **own** public key: one exponentiation per owner, and no interaction between owners at all.

7.4 Tag Generation

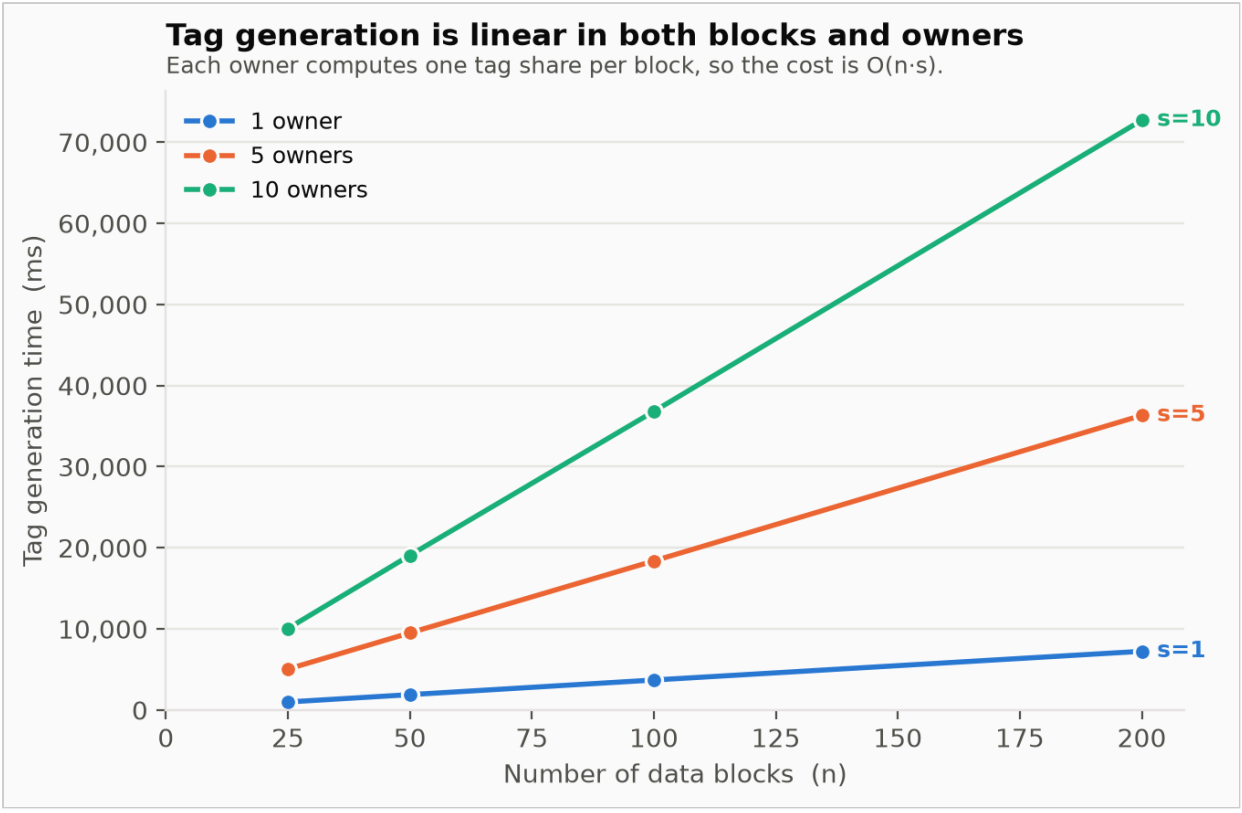


Figure 7.2: Tag generation time versus number of blocks for 1, 5 and 10 owners (reproduces base paper Fig. 5a)

Blocks `n`	s = 1	s = 5	s = 10
25	997 ms	5,051 ms	10,049 ms
50	1,901 ms	9,512 ms	19,076 ms
100	3,706 ms	18,387 ms	36,821 ms
200	7,245 ms	36,326 ms	72,722 ms

Tagging is O(n·s), and the measurements show this almost exactly. At `n = 200`, the three owner counts stand in the ratio

```
1 : 5.01 : 10.04     against an ideal 1 : 5 : 10
```

and doubling `n` from 100 to 200 multiplies the cost by 1.96×, 1.98× and 1.97× for the three series respectively.

This is the one phase where the baseline is asymptotically better: it achieves O(n) by outsourcing signing authority to a trusted third party. The base paper argues (§VII.B) that this

is not a realistic trade: it requires owners to authorise a third party to sign a file that the third party cannot see. The proposed scheme's O(n·s) is the honest price of every owner signing with its own key, and for a single owner it reduces to O(n) regardless.

Tag generation is also a **one-time** cost per dataset, whereas auditing recurs indefinitely.

7.5 Auditing

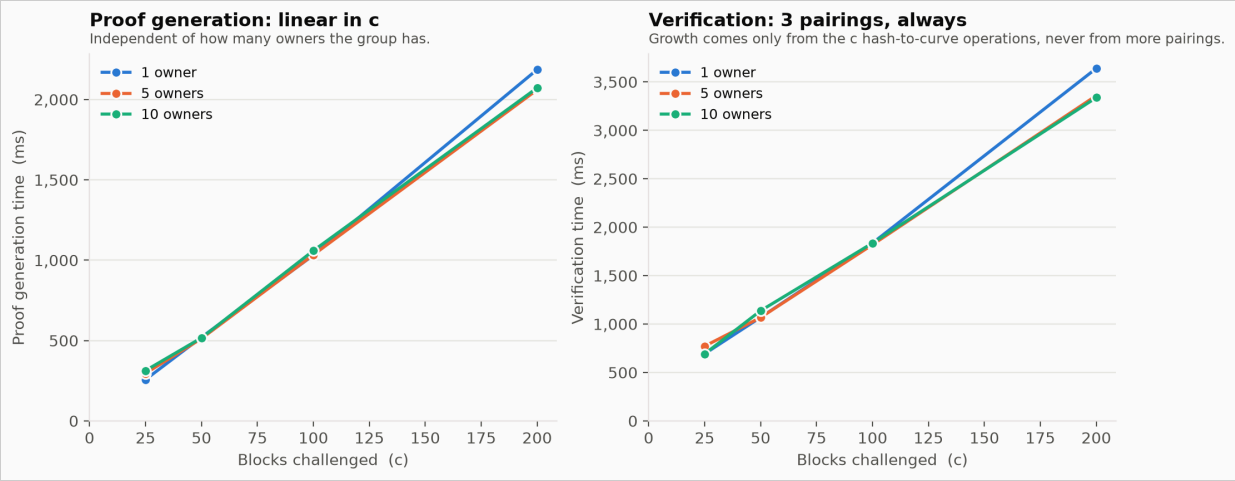


Figure 7.3: Audit proof generation and verification time versus challenge size (reproduces base paper Figs. 5b and 5c)

Challenged `c`	Proof generation	Verification
25	257 ms	693 ms
50	521 ms	1,069 ms
100	1,031 ms	1,836 ms
200	2,186 ms	3,640 ms

7.5.1 Independence from group size: a clean confirmation

The most striking single measurement in this project:

Challenged `c`	1 owner	10 owners	Difference
25	692.6 ms	692.1 ms	**0.07 %**
100	1,835.9 ms	1,835.6 ms	**0.02 %**

Verification cost is **indistinguishable** between a 1-owner and a 10-owner group. Once the tag shares are aggregated, the auditor genuinely cannot tell how many owners produced them. The same holds for proof generation.

This matters practically: an auditor's workload does not grow as consortia grow, so the scheme supports arbitrarily large ownership groups at no cost to the party doing the verifying.

7.5.2 Constant pairing count

Verification performs exactly **three pairings** regardless of `n` or `c`. All growth in the table above comes from the `c` hash-to-curve operations needed to rebuild `∏ H₃(F‖j)^{v_j}`, at 4.8 ms each, `c = 200` accounts for roughly 960 ms of the 3,640 ms measured, with the batched pairings contributing a fixed ~400 ms.

The proof itself is **constant size**: one scalar plus one group element, about 128 bytes, whether 25 or 200 blocks were challenged. Auditing a 100 MB dataset and a 100 TB dataset cost the auditor identical bandwidth.

7.5.3 Detection probability: measured against theory

500 trials per configuration, at `n = 200`:

Corrupted	`c`	Paper's bound	Exact (hypergeometric)	Measured
1 %	100	63.40 %	75.13 %	**73.00 %**
1 %	150	77.85 %	93.84 %	**92.20 %**
1 %	190	85.19 %	99.77 %	**99.80 %**
5 %	50	92.31 %	94.79 %	**96.00 %**
5 %	100	99.41 %	99.92 %	**99.80 %**
10 %	20	87.84 %	89.15 %	**90.20 %**
10 %	50	99.48 %	99.77 %	**99.80 %**

A finding. The base paper uses the classical PDP bound `1 - ((n-t)/n)^c`, which models sampling **with replacement**. The implementation's auditor draws **distinct** block indices, for which the correct model is hypergeometric, `1 - C(n-t, c)/C(n, c)`.

Measured rates track the hypergeometric model closely and sit **at or above the paper's bound in every configuration tested**. The paper's bound is therefore *conservative*, never optimistic: its recommended `c = 460` under-promises what the implementation actually delivers. This is a favourable result for the paper, and it means the recommended parameter can be adopted with confidence.

7.6 Ownership Modification

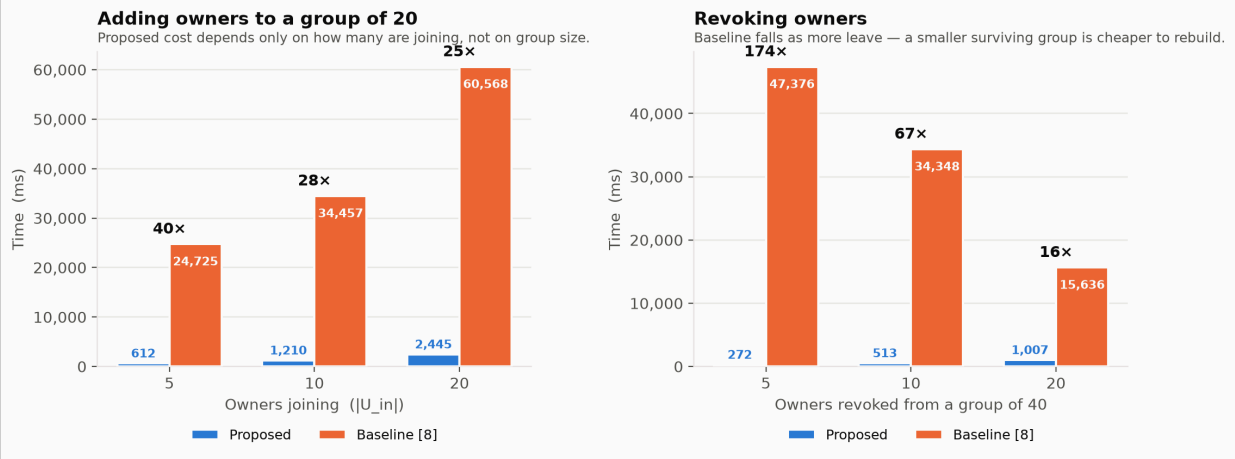


Figure 7.4: Ownership modification cost — owner addition and revocation, proposed against baseline (reproduces base paper Figs. 7a and 7b)

7.6.1 Adding owners (base group of 20)

Joining `|U_in|`	Proposed	Baseline [8]	Speedup
5	612 ms	24,725 ms	**40×**
10	1,210 ms	34,457 ms	**28×**
20	2,445 ms	60,568 ms	**25×**

The proposed cost is almost exactly linear in the number of owners **joining** (612 → 1,210 → 2,445 ms for 5 → 10 → 20 joiners) and does not depend on how large the group already is. Existing owners are not re-keyed and need not even be online.

The baseline must rebuild every authenticator in the enlarged group, because `APK` has changed.

7.6.2 Revoking owners (base group of 40)

Revoked `|U_out|`	Proposed	Baseline [8]	Speedup
5	272 ms	47,376 ms	**174×**
10	513 ms	34,348 ms	**67×**
20	1,007 ms	15,636 ms	**16×**

The two columns move in **opposite directions**, which is the clearest illustration of the difference between the schemes:

- **Proposed** cost *rises* with the number revoked (272 → 1,007 ms), because work is proportional to who is leaving.
- **Baseline** cost *falls* (47,376 → 15,636 ms), because revoking more owners leaves a *smaller* surviving group, and its cost is quadratic in the survivors.

The base paper predicts exactly this behaviour in §VII.B.4. Revoking 5 owners from a group of 40 (a routine administrative action) costs the baseline over 47 seconds against the proposed scheme's 272 milliseconds.

7.7 Tag Update at the Cloud

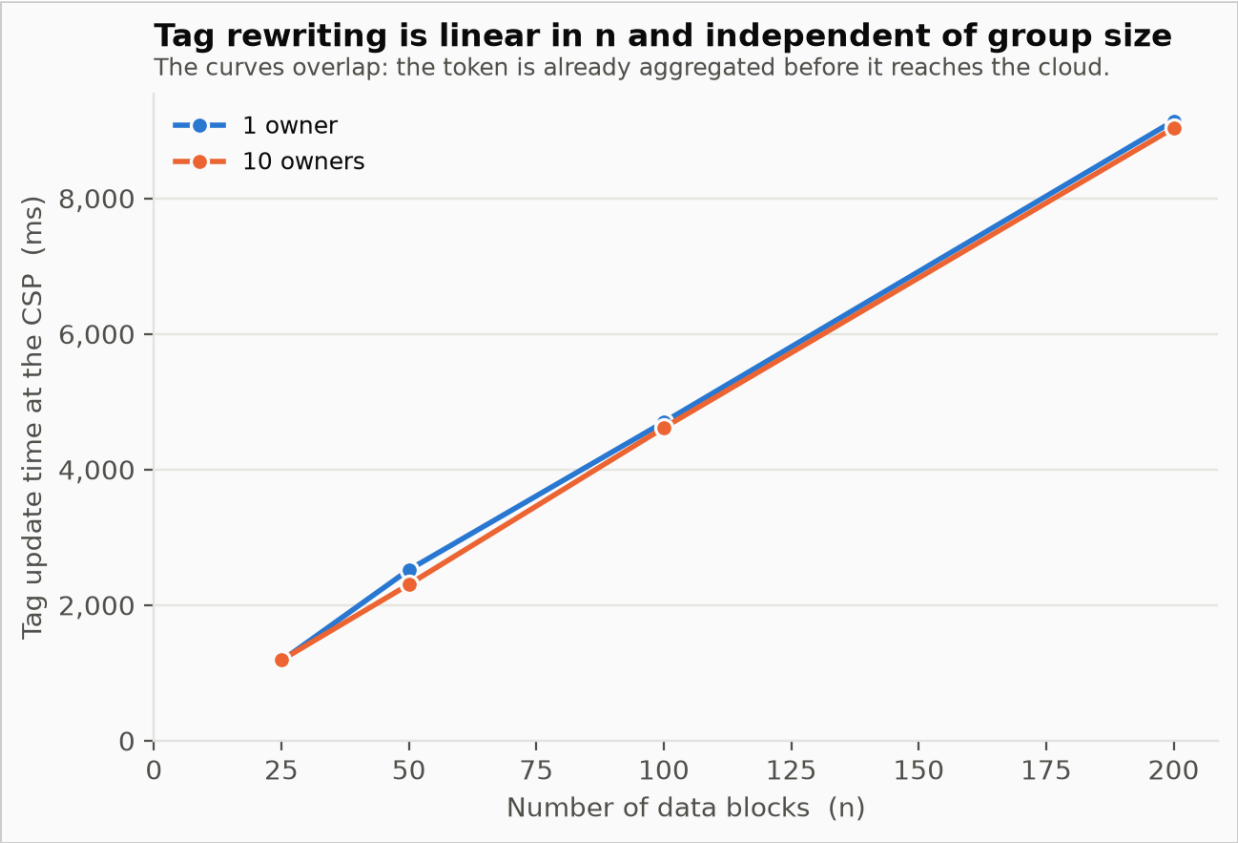


Figure 7.5: Tag update time at the cloud versus number of blocks (reproduces base paper Fig. 7c)

Blocks n	s = 1	s = 10
25	1,178 ms	1,189 ms
50	2,521 ms	2,305 ms
100	4,699 ms	4,616 ms
200	9,140 ms	9,036 ms

Two properties are confirmed. Tag rewriting is **linear in n** (doubling `n` doubles the time), and it is **independent of group size**, the two columns agree to within 2 % throughout, because the ownership token reaching the cloud has already been aggregated.

This is the design placing the only O(n) step of an ownership change on the CSP, the entity the system model describes as having "abundant storage and computational resources", while the owners exchange only a constant-size token.

7.8 Ownership Transfer

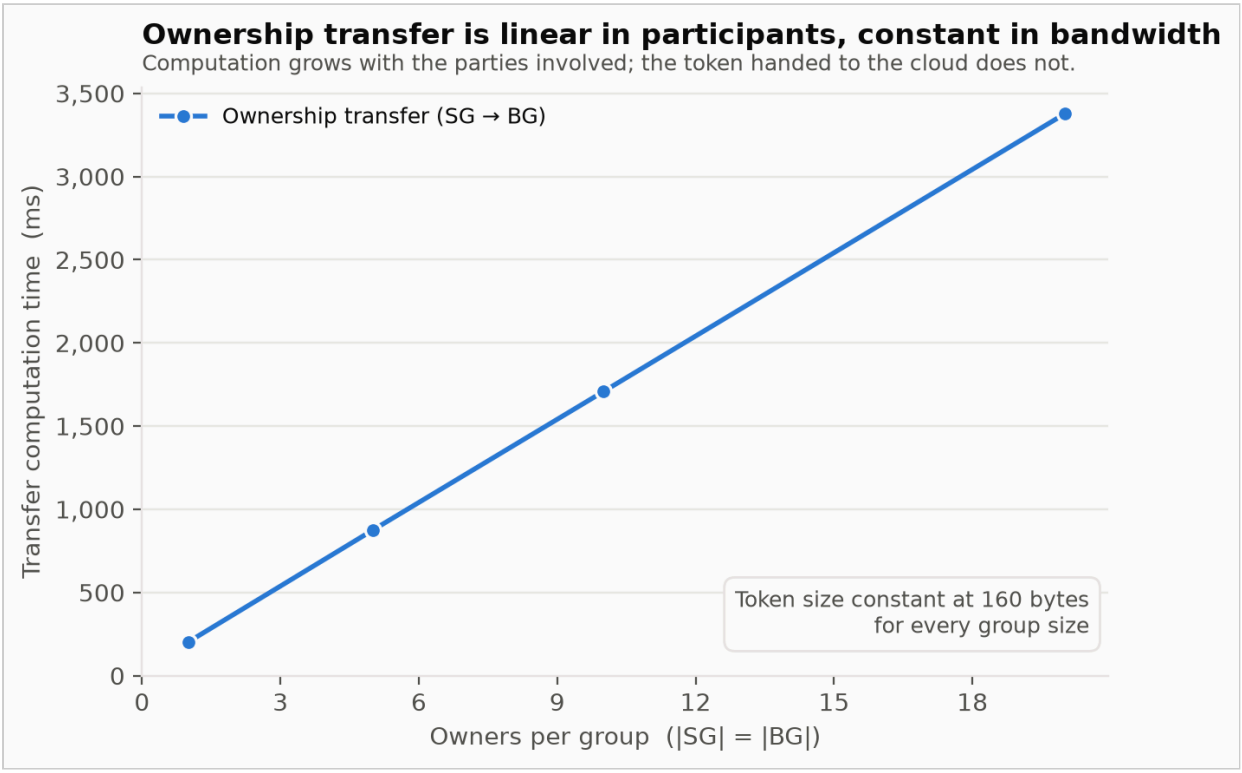


Figure 7.6: Ownership transfer computation and constant token size (reproduces base paper Fig. 8)

Owners per group	Computation	Token size
1	202 ms	160 bytes
5	874 ms	160 bytes
10	1,707 ms	160 bytes
20	3,377 ms	160 bytes

Computation grows linearly with the number of participating owners, since each contributes one share. The **token handed to the cloud is constant at 160 bytes** (two 32-byte scalars and one 96-byte group element) for every group size tested.

This confirms the communication claim of the paper's Table III. Selling a dataset between two twenty-member consortia requires the same 160 bytes of protocol traffic to the cloud as a transfer between two individuals, and the same as it would for a dataset of any size.

7.9 Security Results

Reported in full in Chapter 6; summarised here for completeness.

Attack	Prior art	Proposed scheme
Collusion (buyer + cloud, leaked index secret)	**BROKEN**, forged proof accepted for a deleted block	**RESISTED**
Rogue-key (fake co-ownership)	**BROKEN**, plain aggregate BLS accepted the forgery	**RESISTED**
Data tampering / deletion	—	**Detected in every trial**

The collusion demonstration granted the attacker *more* information than the protocol would ever leak (every owner's `H₂(sk_i‖a_i)` value) and the forgery still failed, because the random masks `a_i` remain bound to the original block index and cannot be recovered from the published `Rs`.

7.10 Discussion of Absolute Performance

The absolute figures in this chapter are substantially slower than those in the base paper, which reports microseconds where this implementation reports milliseconds. The reason is the deliberate choice of `py_ecc`, a **pure-Python** library, over the C implementation with the PBC library used by the paper's authors.

That choice was made on grounds of reproducibility. Two faster alternatives were evaluated and rejected: `blspy` installs successfully but exposes no scalar multiplication on G_1 or G_2, making it structurally unable to implement a custom scheme; `petrelic` has no distribution for the target architecture. A project an examiner cannot run is worth less than a slow one they can.

Three points bound the significance of this gap:

1. **The claim under test is asymptotic.** Both schemes are measured on identical primitives, so the ratio between them (which is what $O(s)$ versus $O(s^2)$ asserts) is unaffected by the constant factor.
2. **Verification is already practical.** At 400–700 ms per audit, and with cost independent of dataset size, the scheme is usable at this speed for periodic auditing of arbitrarily large

datasets.

3. **The gap is addressable in one file.** All pairing operations are confined to `crypto/pairing.py`; substituting native bindings would leave every other module unchanged.

Note on the baseline. All baseline figures are a **cost-model re-implementation** of scheme [8]'s structure as characterised in Table II of the base paper, not the original authors' code. The asymptotic shape is faithful; the absolute constants are ours.

7.11 Summary of Results

Claim under test	Result
KeyGen is O(s), baseline O(s^2)	**Confirmed**, 1.99× vs 3.81× on doubling `s`; 10.5× speedup at s = 20
Membership changes depend only on who is changing	**Confirmed**, up to 174× faster than baseline
Auditing is independent of group size	**Confirmed**, 0.07 % difference between 1 and 10 owners
Verification uses a constant number of pairings	**Confirmed**, 3 pairings for all `n`, `c`
The ownership token is constant size	**Confirmed**, 160 bytes for all group sizes
Corruption is detected at the stated rate	**Confirmed**, and the paper's bound shown to be conservative
Collusion and rogue-key attacks are prevented	**Confirmed**, both break the prior art and both fail here

Every performance and security claim of the base paper that falls within the scope of this project was reproduced.

CHAPTER 8

CONCLUSION

Cloud computing has enabled large-scale data storage and sharing across organizations, creating a growing demand for secure data transactions. Provable Data Possession (PDP) schemes allow users to verify the integrity of cloud-stored data without downloading the entire dataset. However, existing DT-PDP schemes mainly support single-owner transfers and lack efficient support for multi-owner group transactions. Although BLS aggregate signatures facilitate multi-ownership, they introduce scalability and security challenges. To address these limitations, this project proposes a secure and scalable cloud auditing system that enables efficient ownership transfer and integrity verification for group-based data transactions.

The cloud auditing systems that mainly support single-owner environments, the proposed framework effectively supports multi-owner data sharing, dynamic ownership modification, and secure ownership transfer between groups. The proposed system enhances security by mitigating threats including collusion attacks, key leakage, unauthorized access, and data tampering. Additionally, the incorporation of a Third-Party Auditor (TPA) facilitates efficient data integrity verification, thereby minimizing the overhead on data owners.

REFERENCES

[1] C. Wu, W. You, and X. Huang, "Scalable Cloud Auditing With Efficient Ownership Transfer for Group Transactions" [Base Paper], *IEEE Transactions on Information Forensics and Security*, vol. 20, 2025. https://doi.org/10.1109/TIFS.2025.3636056

[2] G. Ateniese et al., "Provable Data Possession at Untrusted Stores," *ACM CCS*, 2007. https://dl.acm.org/doi/10.1145/1315245.1315318

[3] A. Juels and B. Kaliski, "PORs: Proofs of Retrievability for Large Files," *ACM CCS*, 2007. https://dl.acm.org/doi/10.1145/1315245.1315317

[4] H. Shacham and B. Waters, "Compact Proofs of Retrievability," *Journal of Cryptology*. https://link.springer.com/article/10.1007/s00145-012-9129-2

[5] C. Wang, Q. Wang, K. Ren, and W. Lou, "Privacy-Preserving Public Auditing for Secure Cloud Storage." https://doi.org/10.1109/TC.2011.245

[6] L. Rao, H. Zhang, and T. Tu, "Dynamic Outsourced Auditing Services for Cloud Storage Based on Batch-Leaves-Authenticated Merkle Hash Tree." https://doi.org/10.1109/TSC.2017.2708116

[7] H. Tian, Y. Chen, C.-C. Chang, H. Jiang, and Y. Huang, "Dynamic-Hash-Table Based Public Auditing for Secure Cloud Storage." https://doi.org/10.1109/TSC.2015.2512589

[8] J. Li, H. Yan, and Y. Zhang, "Efficient Identity-Based Provable Multi-Copy Data Possession in Multi-Cloud Storage." https://doi.org/10.1109/TCC.2019.2929045

[9] B. Wang, H. Li, X. Liu, F. Li, and X. Li, "Efficient Public Verification on the Integrity of Multi-Owner Data in the Cloud." https://doi.org/10.1109/JCN.2014.000105

[10] Y. Zhang, J. Yu, R. Hao, C. Wang, and K. Ren, "Enabling Efficient User Revocation in Identity-Based Cloud Storage Auditing for Shared Big Data." https://doi.org/10.1109/TDSC.2018.2829880

[11] Q. Wang, C. Wang, K. Ren, and W. Lou, "Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing." https://doi.org/10.1109/TPDS.2010.183

[12] T. Kavitha, Sk. Himam Basha, K. Winshiny Madonna, K. Sireesha, Tufail M.S, and M. Odeh, "Experimental Evaluation of Secure and Verifiable Data Control Access over Cloud Storage Environment using Dynamic Cryptographic Strategy." https://doi.org/10.1109/ICSES63760.2024.10910755

[13] J. Li, H. Yan, and Y. Zhang, “Identity-Based Privacy Preserving Remote Data Integrity Checking for Cloud Storage.” https://doi.org/10.1109/JSYST.2020.2978146

[14] A. Joshi, A. Dumka, A. Raturi, D. P. Singh, and S. Kumar, “Improved Security and Privacy in Cloud Data Security and Privacy: Measures and Attacks.” https://doi.org/10.1109/ICFIRTP56122.2022.10063186

[15] W. Shen, C. Gai, J. Yu, and Y. Su, “Keyword-Based Remote Data Integrity Auditing Supporting Full Data Dynamics.” https://doi.org/10.1109/JIOT.2023.3282939

[16] C. Gai, W. Shen, M. Yang, and J. Yu, “PPADT: Privacy-Preserving Identity-Based Public Auditing With Efficient Data Transfer for Cloud-Based IoT Data.” https://doi.org/10.1109/TSC.2023.3339521

[17] B. Wang, B. Li, and H. Li, “Privacy-Preserving Public Auditing for Shared Data in the Cloud.” https://doi.org/10.1109/TCC.2014.2299807

[18] A. F. Barsoum and M. A. Hasan, “Provable Multicopy Dynamic Data Possession in Cloud Computing Systems.” https://doi.org/10.1109/TIFS.2014.2384391

[19] H. Wang, D. He, A. Fu, Q. Li, and Q. Wang, “Provable Data Possession with Outsourced Data Transfer.” https://doi.org/10.1109/TSC.2019.2892095

[20] C. Siva Kumar, P. Jagruthi, L. Vasantha, N. Pavithra, P. Siroshini, and T. Supriya, “Secure and Proficient Provable Data Procurity With Privacy Protection in Cloud Storage.” https://doi.org/10.1109/ICCCI56745.2023.10128477

APPENDIX

Justification for SDG Mapping

Cloud-based auditing system with efficient ownership management for group data transaction

Project Details

Team No.	28
Team Members	Sanjay GP (1GA23IS141), Shashank Gowda TK (1GA23IS144), Srujangouda Malipatil (1GA23IS165), Yashwanth Gowda DK (1GA23IS188)
Domain	Cloud Computing
Project Type	Research
Factors Related	Safety

SDG Justification

This project supports SDG 9 by improving secure and reliable cloud-based infrastructure. It promotes innovation through advanced auditing and ownership management techniques. The system enhances data security, efficiency, and scalability, contributing to the development of modern digital infrastructure.

Mapped Sustainable Development Goals: SDG 9 — Industry, Innovation and Infrastructure